

Biologically Inspired Artificial Intelligence

Klasyfikacja i rekonstrukcja obrazów z sygnału EEG

Raport z projektu

Paulina Kędzierska
Szymon Konarski

22.08.2026

1. Cel projektu

Celem projektu było utworzenie sieci neuronowej która byłaby w stanie sklasyfikować i odtworzyć obrazy wyłącznie na podstawie sygnału EEG. W tym celu zapoznano się z typowym przebiegiem badania EEG, strukturą plików wynikowych oraz typowymi problemami występującymi podczas takiego badania. Następnie dokonano analizy danych i odpowiednio przygotowano je do szkolenia kilku modeli sieci neuronowych, aby przeprowadzić próby klasyfikacji oraz rekonstrukcji obrazów. Cały przebieg projektu, poczynione obserwacje oraz wyciągnięte wnioski zostały przedstawione w niniejszym raporcie.

2. Założenia

2.1. Struktura danych

W badaniu wykorzystano obrazy należące do 11 kategorii: abstrakcja, samolot, jabłko, banan, ptak, łódź, samochód, pies, osoba, pociąg oraz zebra. Kategoria abstrakcja różniła się od pozostałych, ponieważ zamiast zdjęć zawierała prostą grafikę złożoną z figur geometrycznych.

W każdej kategorii znajdowało się 20 obrazów, co dawało łącznie 220 unikalnych obrazów. Zestaw ten był prezentowany w sześciu seriach, więc każdy obraz pojawiał się podczas badania sześć razy. W efekcie dla jednego badanego otrzymano łącznie 1320 prób, czyli po 220 prób w każdej serii.

Każda próba miała przypisany numer serii oraz numer próby. W pliku z zapisanymi zdarzeniami znajdowały się również informacje o czasie wystąpienia danego zdarzenia oraz jego rodzaju. Dzięki temu można było później określić, kiedy dokładnie pojawił się konkretny obraz i jaki fragment sygnału EEG odpowiadał jego prezentacji.

Takie uporządkowanie danych było szczególnie istotne z punktu widzenia dalszej analizy. Pozwalało ono połączyć sygnał EEG z konkretnym obrazem i jego kategorią, a także rozdzielić poszczególne etapy pojedynczej próby.

2.2. Przebieg badania

Przed rozpoczęciem właściwego badania uczestnik uzupełniał krótką ankietę oraz otrzymał instrukcję dotyczącą przebiegu eksperymentu. Następnie wykonano krótki etap przygotowawczy, w którym rejestrowano aktywność EEG przy zamkniętych oczach, a później przy otwartych oczach. Po tym etapie rozpoczęła się właściwa część badania.

Badanym był student realizujący projekt, który przed rozpoczęciem eksperymentu nie miał dostępu do obrazów wykorzystywanych podczas badania. Uczestnik miał założony czepek EEG z elektrodami, a jego zadaniem było skupienie wzroku na wyświetlanych obrazach i późniejsze ich opisanie.

Każda próba przebiegała według tego samego schematu:

*fiksacja → czarny ekran → obraz → czarny ekran → opis obrazu → naciśnięcie spacji
→ czarny ekran*

Na początku pojawiał się punkt fiksacji, na którym badany miał skupić wzrok. Następnie na krótko wyświetlany był czarny ekran, po którym pojawiał się właściwy obraz. Obraz był prezentowany przez około 0,5 sekundy.

Po obejrzeniu każdego obrazu uczestnik miał jednym słowem opisać to, co zobaczył. Kiedy kończył opis, naciskał klawisz spacji. W tym momencie zapisywane było zdarzenie `SPACE_PRESSED`, a następnie pojawiał się krótki czarny ekran i rozpoczynała się kolejna próba.

Poszczególne etapy były zapisywane w pliku z wykorzystaniem specjalnych znaczników. Przykładowo, `FIXATION` oznaczało rozpoczęcie fiksacji, `IMAGE_ON` moment pojawienia się obrazu, `DESCRIBE_SCREEN` rozpoczęcie etapu opisu, a `SPACE_PRESSED` zakończenie opisu przez badanego. Dzięki temu późniejsza analiza nie wymagała zgadywania, w którym momencie prezentowany był dany obraz.

Całe badanie zostało podzielone na sześć serii po 220 prób. Pomiędzy seriami uczestnik miał przerwy, dzięki czemu mógł odpocząć przed rozpoczęciem kolejnej części badania. Łączny czas rejestracji wyniósł około 2,5 godziny, wliczając przygotowanie, przerwy oraz poszczególne etapy badania.

Czas potrzebny na wykonanie pojedynczej próby nie był zawsze taki sam. Sama prezentacja obrazu trwała około 0,5 sekundy, natomiast etap opisywania zależał od badanego i od tego, ile czasu potrzebował on na przygotowanie odpowiedzi. Z tego powodu poszczególne serie trwały różną ilość czasu.

Po zakończeniu głównej części eksperymentu przeprowadzono również etap końcowy, obejmujący ankietę po badaniu. Pozwoliło to zbierać dodatkowe informacje dotyczące udziału w eksperymencie, niezwiązane bezpośrednio z samym zapisem EEG.

W rezultacie otrzymano zbiór danych zawierający zarówno sygnał EEG, jak i informacje o dokładnym przebiegu eksperymentu. Dzięki zapisanym znacznikom zdarzeń możliwe jest określenie, kiedy uczestnik oglądał konkretny obraz, kiedy go opisywał oraz kiedy zakończył daną próbę. Stanowi to podstawę do dalszego przetwarzania i analizy zebranych danych.

3. Użyte środowisko programistyczne, biblioteki i narzędzia

3.1. Środowisko programistyczne

Projekt realizowano w kilku środowiskach. Kod tworzone i uruchamiano głównie w środowisku PyCharm oraz w terminalu systemu Linux (Ubuntu) – po zainstalowaniu Pythona 3 wraz z potrzebnymi pakietami i bibliotekami. Część prac prowadzono również w programie PowerShell na systemach Windows 10 i 11 (analogicznie, po zainstalowaniu Pythona 3.11

i wymaganych bibliotek) oraz w środowisku Google Colab, wygodnym zwłaszcza do uczenia sieci neuronowych ze względu na dostęp do akceleracji GPU.

3.2. Biblioteki

W projekcie wykorzystano przede wszystkim biblioteki języka Python: MNE (wczytywanie i przetwarzanie sygnału EEG – filtrowanie, montaż, analiza ICA, tworzenie epok), NumPy oraz pandas (operacje na tablicach danych i metadanych), Matplotlib (wizualizacje: wykresy, macierze pomyłek, siatki obrazów), a także scikit-learn (kodowanie etykiet, podział danych oraz miary skuteczności, m.in. raport klasyfikacji i macierz pomyłek). Do budowy i uczenia sieci neuronowych użyto biblioteki TensorFlow wraz z interfejsem Keras, natomiast do wczytywania i skalowania obrazów na potrzeby rekonstrukcji – biblioteki Pillow.

3.3. Narzędzia

Do przygotowania diagramów potoku przetwarzania danych wykorzystano program draw.io (diagrams.net). W pracy nad kodem oraz analizą wyników pomocniczo korzystano z narzędzi generatywnej sztucznej inteligencji (m.in. Claude, ChatGPT oraz Gemini). Oprócz gotowych narzędzi projekt obejmował również zestaw własnych skryptów w języku Python, odpowiedzialnych za kolejne etapy pracy: przygotowanie danych, mieszanie i podział zbiorów, uczenie modeli oraz rekonstrukcję obrazów.

3.4. Sieci neuronowe

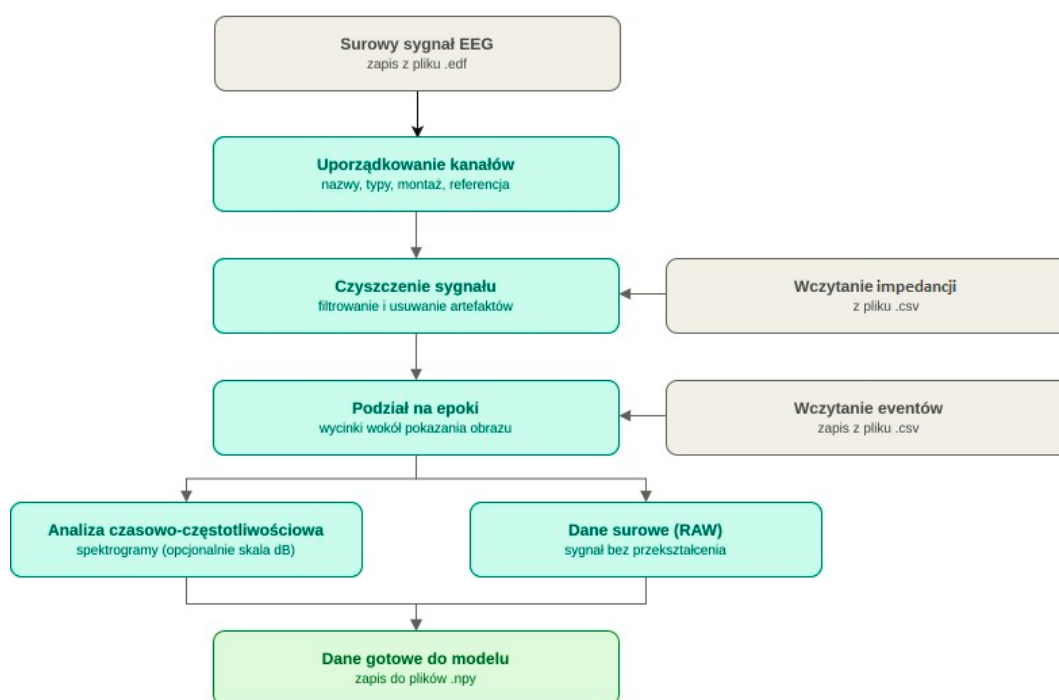
W zadaniu klasyfikacji obrazów wykorzystano finalnie trzy architektury sieci konwolucyjnych: klasyczną sieć CNN działającą na reprezentacji czasowo-częstotliwościowej (spektrogramach) oraz sieci EEGNet i EEG-Inception, operujące na surowym sygnale w dziedzinie czasu. Wszystkie trzy porównano na tym samym zadaniu i podziale danych.

W zadaniu rekonstrukcji obrazów zastosowano autoenkoder, którego enkoder oparto na architekturze EEGNet, a dekodery zbudowano ze splotów transponowanych, rozwijających wektor cech z powrotem do obrazu. W uzupełniającym wariancie opartym na identyfikacji obrazów (ang. *retrieval*) do wyznaczenia reprezentacji obrazów wykorzystano dodatkowo pretrenowaną sieć MobileNetV2.

Architektura EEGNet pełniła zatem rolę wspólnego rdzenia – stosowano ją zarówno w klasyfikacji, jak i jako enkoder w zadaniach rekonstrukcji. Na etapie prac porównywano ponadto różne reprezentacje sygnału (surową oraz czasowo-częstotliwościową), których wpływ na skuteczność omówiono w sekcji poświęconej klasyfikacji.

4. Przygotowanie danych

Przygotowanie danych zrealizowano w postaci potoku przetwarzania (ang. *pipeline*), złożonego z szeregu funkcji wykonywanych w ustalonej kolejności. Każda z nich odpowiadała za pojedynczy etap obróbki sygnału – od wczytania surowego zapisu, przez porządkowanie i czyszczenie danych, aż po podział na próby i zapis w postaci gotowej do wykorzystania przez model.



Rys. 1. Ogólny schemat potoku przygotowującego dane do nauki sieci neuronowej.

Poniżej opisano finalną postać potoku oraz wskazano w jaki sposób zmieniał się on w trakcie realizacji projektu.

4.1. Parametry skryptu

Na początku skryptu zgromadzono wszystkie parametry sterujące przebiegiem przetwarzania. Takie rozwiązanie pozwalało uruchamiać potok w różnych wariantach – m.in. dla wybranych badanych czy z różnymi metodami normalizacji – bez konieczności ingerencji w dalszą część kodu.

```

now = datetime.now().strftime("%m%d_%H%M")

nicki_badanych = ["abc", "Bear", "fghx", "jt", "mi2", "miguel", "mole", "Reshi", "sapling"]
csv_eventow = ["abc_EEGBasedVisualRecall_Events_Rep1_2026-05-27_11-04-56",
               "Bear_EEGBasedVisualRecall_Events_Rep1_2026-05-29_10-34-33",
               "fghx_EEGBasedVisualRecall_Events_Rep1_2026-05-27_14-21-40",
               "jt_EEGBasedVisualRecall_Events_Rep1_2026-06-10_10-07-50",
               "MI2_EEGBasedVisualRecall_Events_Rep1_2026-06-18_11-42-39",
               "miguel_EEGBasedVisualRecall_Events_Rep1_2026-06-11_09-42-59",
               "mole_EEGBasedVisualRecall_Events_Rep2_2026-05-22_11-27-28",
               "Reshi_EEGBasedVisualRecall_Events_Rep1_2026-05-28_09-40-36",
               "sapling_EEGBasedVisualRecall_Events_Rep1_2026-06-12_09-43-56"]
reduction_methods = ["ratio", "logratio", "zlogratio", "mean", "zscore"]
categories=["abstract","airplane","apple","banana","bird","boat","car","dog","person","train","zebra"]

# Wybór obecnych badanych
wybrani_badani = [0]

# Opcje edycji epok
do_morlet = True      #Czy dane po TFA czy surowe?
morlet_log_scale = True
subtract_mean_baseline = False

reduct_tfa_using_baseline = False
chosen_reduction_method = reduction_methods[2]

# Opcje zapisu do .npy
do_save_to_file = True

# Opcje wizualizacji
visualization_catalogue="vis"
image_channel = 14
image_epoch = 14
do_generate_single_image = False

chosen_category = "zebra"
do_generate_single_gif_chrono_order = False
do_generate_single_gif_image_order = False
do_generate_combo_gif = False

# Zmienne globalne
obecnie_badani = [nicki_badanych[i] for i in wybrani_badani]
NICK_BADANEGO = nicki_badanych[wybrani_badani[0]]
CSV_EVENTY = csv_eventow[wybrani_badani[0]]
try:
    current_category = categories.index(chosen_category)
except ValueError:
    current_category = 0

raw_data: mne.io.Raw
epochs: mne.Epochs
tfData_float32: np.float32

```

Rys. 2. Parametry skryptu.

Podstawę stanowiły listy opisujące dostępne dane. W zmiennej `nicki_badanych` zapisano identyfikatory (pseudonimy) poszczególnych uczestników, natomiast w `csv_eventow` – odpowiadające im nazwy plików ze zdarzeniami. Obie listy miały tę samą kolejność, dzięki czemu danego badanego oraz jego plik zdarzeń wskazywano tym samym indeksem. Ponadto w zmiennej `categories` przechowywano jedenaście kategorii obrazów wykorzystanych

w badaniu, a w `reduction_methods` – dostępne metody normalizacji sygnału względem linii bazowej (m.in. *logratio* czy *zscore*). Pomocniczo wyznaczano również znacznik czasu (`now`), wykorzystywany przy nazywaniu plików wynikowych.

Zmienna `wybrani_badani` określała, którzy uczestnicy mają zostać przetworzeni w danym uruchomieniu; wskazywano ich poprzez indeksy na wcześniejszych listach. Na etapie dopracowywania potoku ograniczano się zwykle do jednego badanego, co skracało czas obliczeń.

Kolejna grupa parametrów odpowiadała za sposób przygotowania epok. Flaga `do_morlet` decydowała o tym, czy dane poddawane były analizie czasowo-częstotliwościowej, czy przekazywane w postaci surowej. Parametr `morlet_log_scale` włączał przeliczenie mocy na skalę logarytmiczną, `subtract_mean_baseline` – odjęcie średniej liczonej pomiędzy kanałami, a `reduct_tfa_using_baseline` wraz z parametrem `chosen_reduction_method` określały, czy i w jaki sposób sygnał odnoszony był do linii bazowej. Osobna flaga `do_save_to_file` decydowała o zapisaniu przygotowanych danych do plików.

Wydzielono również ustawienia dotyczące wizualizacji, służące wyłącznie do podglądu danych na etapie eksploracji. Zmienne `image_channel`, `image_epoch` oraz `chosen_category` wskazywały, który kanał, którą epokę i którą kategorię obrazu przedstawić, natomiast zestaw flag `do_generate_...` pozwalał włączać poszczególne rodzaje ilustracji (pojedynczy obraz oraz animacje w różnym porządku). Katalog docelowy dla tych plików określała zmienna `visualization_catalogue`.

Na koniec, na podstawie powyższych ustawień, wyznaczano zmienne pomocnicze wykorzystywane w dalszej części kodu – m.in. listę wybranych uczestników (`obecnie_badani`) oraz pseudonim i plik zdarzeń pierwszego z nich (`NICK_BADANEGO`, `CSV_EVENTY`). Zadeklarowano także zmienne globalne przechowujące kolejne postacie danych – surowy zapis (`raw_data`), sygnał podzielony na epoki (`epochs`) oraz przetworzoną reprezentację liczbową (`tfData_float32`) – które były współdzielone i modyfikowane przez poszczególne funkcje potoku.

4.2. Wczytanie surowych danych

Pierwszym etapem było wczytanie surowego zapisu EEG. Dane każdego badanego przechowywane były w pliku w formacie EDF (ang. *European Data Format*), będącym standardem stosowanym do zapisu sygnałów biomedycznych.

```

#-----Wczytywanie danych-----
def load_data(nick):
    global raw_data

    raw_data = mne.io.read_raw_edf(f"assets/{nick}.raw.edf", preload=True, infer_types=True) # załaduj do pamięci, typ EEG do znanych kanałów
    print("\n#####\n")
    print(raw_data) #Informacje o kanałach i obiekcie
    print("\n#####\n")
    print(raw_data.info) #Informacje o kanałach i obiekcie
    print("\n#####\n")
    print(raw_data.describe()) #Informacje o kanałach i obiekcie
    print("\n#####\n")

```

Rys. 3. Fragment kodu pipeline’u przygotowującego dane, odpowiedzialny za wczytanie surowych danych do programu.

Do ich odczytu wykorzystano funkcję `read_raw_edf()` z biblioteki MNE. Sygnał wczytano od razu w całości do pamięci operacyjnej (parametr `preload=True`), co było konieczne, aby możliwe było jego późniejsze filtrowanie i modyfikacja. Włączono również automatyczne rozpoznawanie typów kanałów na podstawie ich nazw (parametr `infer_types=True`). Po wczytaniu wyświetlono podstawowe informacje o nagraniu – m.in. liczbę i nazwy kanałów oraz częstotliwość próbkowania – w celu wstępnej weryfikacji poprawności odczytanych danych.

4.3. Mapowanie kanałów

Kolejnym krokiem przygotowania danych było uporządkowanie informacji o kanałach. W tym celu jednolicono ich nazwy, przypisano im odpowiednie typy oraz określono położenie elektrod zgodnie z międzynarodowym standardem 10–20.

Standard 10–20 (ang. *international 10–20 system*) to jeden ze sposobów rozmieszczania i nazywania elektrod na powierzchni głowy. Jego nazwa wynika ze sposobu wyznaczania pozycji elektrod: rozmieszcza się je w odstępach stanowiących około 10% lub 20% odległości pomiędzy charakterystycznymi punktami anatomicznymi głowy – nasion (zagłębienie u nasady nosa), inion (guzowatość potyliczna) oraz punktami przedusznymi po obu stronach głowy. Dzięki temu możliwe jest stosowanie podobnego układu elektrod niezależnie od kształtu i rozmiaru głowy badanego, co ułatwia porównywanie wyników uzyskanych dla różnych osób.

Nazwa każdej elektrody koduje jej położenie. Litera (lub zestaw liter) wskazuje jej położenie na powierzchni głowy:

- Fp – okolice czołowo-biegunową,
- F – czołową,
- C – centralną,
- T – skroniową,
- P – ciemieniową,
- O – potyliczną.

W standardzie mogą występować również oznaczenia pośrednie, takie jak FC, CP czy P0, jednak nie odnotowano ich dla opisywanego badania.

Liczby nieparzyste oznaczają lewą stronę głowy, a parzyste prawą. Im większa liczba, tym dalej od linii środkowej znajduje się elektroda. Litera z (od ang. *zero*) oznacza położenie na linii środkowej. Przykładowo, Fp1 oznacza elektrodę znajdującą się po lewej stronie w okolicy czołowo-biegunowej, Fp2 – jej odpowiednik po prawej stronie, natomiast elektroda Cz leży centralnie na linii środkowej. W niniejszym badaniu szczególnie istotne były elektrody 01 i 02, znajdujące się odpowiednio po lewej i prawej stronie w okolicy potylicznej, ponieważ rejestrowana aktywność była związana m.in. z odpowiedzią kory wzrokowej.

Przygotowanie kanałów przebiegało w kilku krokach, których kod przedstawiono na rys. 4.

```
#-----Zmiana nazw kanałów-----
def change_channel_names():
    global raw_data

    new_names = { ch:ch.split('-')[0].strip() for ch in raw_data.ch_names}
    raw_data.rename_channels(new_names)

#-----Zmiana typów kanałów-----
def change_channel_types():
    global raw_data

    ch_types = {}
    for ch in raw_data.ch_names:
        if ch in ['X1:', 'X2:', 'X3:', 'CM']:
            ch_types[ch] = 'misc'
        elif ch in ['Trigger', 'Event']:
            ch_types[ch] = 'stim'
        elif ch[:3] == "Imp":
            raw_data.drop_channels([ch])
        else:
            ch_types[ch] = 'eeg'
    raw_data.set_channel_types(ch_types)

#-----Zmiana montażu i referencji-----
def change_montage_and_reference():
    global raw_data

    montage = mne.channels.make_standard_montage('standard_1020')
    raw_data.set_montage(montage, on_missing='ignore')

    raw_data.set_eeg_reference(ref_channels=['A1', 'A2'])
```

Rys. 4. Fragment kodu pipeline’u przygotowującego dane, odpowiedzialny za mapowanie kanałów na standard `standard_1020`.

Na początku ujednolicono nazwy kanałów poprzez usunięcie fragmentów znajdujących się po znaku -. W tym celu wykorzystano funkcję `split()`, która pozwoliła pozostawić pierwszą część każdej nazwy. Przykładowo, Fp1-REF zostało zmienione na Fp1. W

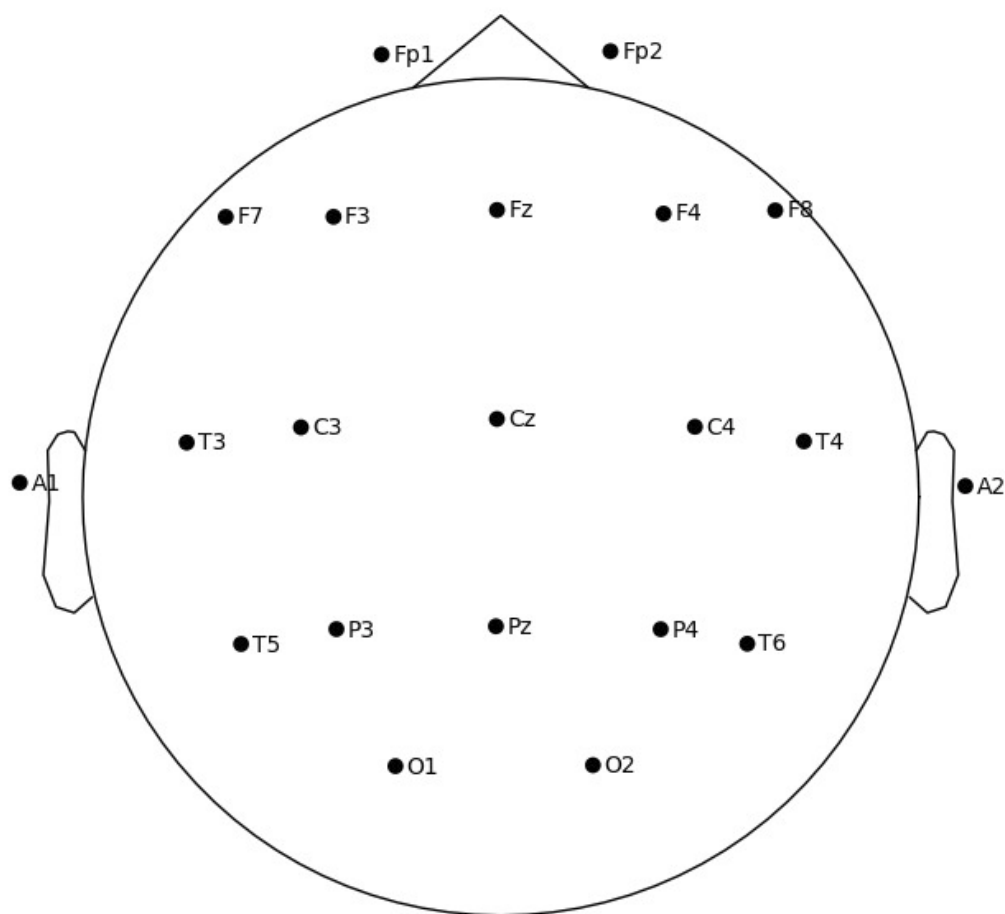
przypadku kanałów zawierających dwukropek, np. `X1:` lub `Ax:`, znak ten pozostawał częścią nazwy.

Następnie każdemu kanałowi przypisano odpowiedni typ. Elektrody EEG oznaczono jako `eeg`, kanały pomocnicze – wejścia analogowe `X1-X3` oraz elektrodę trybu wspólnego `CM` – jako `misc`, a kanały ze znacznikami zdarzeń (`Trigger`, `Event`) jako `stim`. Kanały zawierające pomiary impedancji (o nazwach rozpoczynających się od `Imp`) usunięto, ponieważ nie niosą informacji przydatnej w dalszej analizie.

Podział kanałów na poszczególne typy był istotny podczas dalszego przetwarzania danych, ponieważ kolejne operacje, takie jak montaż, filtrowanie czy analiza ICA, domyślnie obejmują wyłącznie kanały typu `eeg`. Kanały `misc` przechowują dodatkowe informacje, które nie są traktowane jak sygnał mózgowy, natomiast kanały `stim` pozwalały określić moment wystąpienia poszczególnych zdarzeń. Z tego powodu kanały `misc` i `stim` nie zostały wycięte ze zbioru danych, tylko jedynie odpowiednio oznaczone.

Po uporządkowaniu kanałów przypisano do danych montaż `standard_1020`. Wykorzystano w tym celu funkcję `make_standard_montage()`, a następnie `set_montage()`. Montaż zawiera informacje o przestrzennym położeniu elektrod, dzięki czemu nazwy kanałów, mogły zostać powiązane z ich pozycjami na głowie. Parametr `on_missing='ignore'` pozwolił na pominięcie kanałów, dla których nie znaleziono odpowiedniej pozycji w standardzie 10-20, bez zgłaszania błędu.

Poprawność przypisania montażu sprawdzono za pomocą funkcji `plot_sensors(kind='topomap', show_names=True)`. Fragment ten nie znalazł się w funkcji, ponieważ służył typowo do sprawdzenia, czy mapowanie kanałów przebiegło pomyślnie. Wynik przedstawiono na rys. 5.



Rys. 5. Rozmieszczenie elektrod EEG po przypisaniu montażu `standard_1020`.

Na wykresie elektrody EEG zostały rozmieszczone zgodnie ze standardem 10–20. Widoczna jest symetria układu względem linii środkowej głowy oraz podział elektrod na lewą i prawą stronę. Kanały pomocnicze nie są przedstawione jako elektrody na mapie, ponieważ nie mają przypisanych pozycji w zastosowanym montażu. Uzyskany wynik potwierdził poprawność przypisania nazw elektrod do standardowych pozycji przestrzennych. Przygotowane w ten sposób dane mogły zostać wykorzystane w kolejnych etapach analizy.

Ostatnim krokiem tej części przygotowania danych było ustalenie sposobu referencji sygnału. Wartości rejestrowane na poszczególnych elektrodach mają charakter względny, dlatego wymagają odniesienia do wspólnego punktu. Jako referencję przyjęto uśrednienie sygnału z elektrod usznych A1 i A2, co jest jednym z typowych rozwiązań stosowanych w badaniach EEG. Operację tę wykonano funkcją `set_eeg_reference(ref_channels=['A1', 'A2'])`.

4.4. Filtrowanie sygnału

Zarejestrowany sygnał EEG zawiera nie tylko aktywność mózgu, lecz również zakłócenia pochodzące z otoczenia oraz artefakty techniczne.

```
#-----Filtrowanie-----  
def filter_raw_data():  
    global raw_data  
    raw_data = raw_data.notch_filter(freqs=50, verbose=False)  
    raw_data = raw_data.filter(l_freq=1, h_freq=40, verbose=False)
```

Rys. 6. Fragment kodu pipeline'u przygotowującego dane, odpowiedzialny za filtrowanie sygnału.

W celu ich ograniczenia zastosowano dwa filtry. Najpierw użyto filtra zaporowego (ang. *notch filter*) o częstotliwości 50 Hz, usuwającego zakłócenia pochodzące z sieci energetycznej (w Europie napięcie zmienne ma częstotliwość 50 Hz). Następnie zastosowano filtr pasmowoprzepustowy, przepuszczający składowe z zakresu od 1 do 40 Hz. Dolna granica pozwoliła usunąć bardzo wolne wahania sygnału (dryf), natomiast górna – wysokoczęstotliwościowe zakłócenia, w tym część artefaktów mięśniowych. Pozostawione pasmo obejmuje zakres istotny z punktu widzenia analizowanej aktywności kory wzrokowej.

4.5. Wykrywanie i oznaczanie kanałów o niskiej jakości

Podczas rejestracji jakość sygnału na poszczególnych elektrodach może się różnić, m.in. ze względu na niedostateczny kontakt elektrody ze skórą. Informacje o jakości połączenia zapisane były w osobnym pliku, zawierającym dla kolejnych chwil czasowych oznaczenia stanu każdego kanału. Na tej podstawie zastosowano dwustopniowy mechanizm oceny.

```

#-----Automatyczne wykrywanie złych kanałów na podstawie impedancji-----
def delete_bad_channels():
    # 1. Kanały z bardzo dużym udziałem Low_SNR oznaczamy jako "bad"
    # 2. Dla pozostałych kanałów tworzymy adnotacje czasowe BAD_*
    # aby odrzucać tylko fragmenty nagrania o złej jakości
    df_imp = pd.read_csv(f"assets/{NICK_BADANEGO}_imp.csv", skiprows=6)

    bad_channels = []
    annotations = []

    excluded_imp_channels = ["X1:", "X2:", "X3:", "CM"]          # Kanały pomocnicze - nie analizujemy ich jakości EEG
    dead_channel_threshold = 0.50                               # Kanał uznajemy za martwy jeśli >50% próbek ma Low_SNR

    time_step = df_imp["Time"].diff().median()                 # Maksymalna przerwa (w sekundach) między kolejnymi wpisami Low_SNR,
                                                                # aby traktować je jako jeden ciągły fragment
    merge_gap = time_step * 1.5                                # merge_gap = 1.5 # bezpieczna, stała wartość,
                                                                # gdy kolumna Time jest właśnie próbkowana co 1 sekundę

    for col in df_imp.columns:
        if col == "Time":
            continue
        if col in excluded_imp_channels:
            print(f"{col}: pominięty (kanał pomocniczy)")
            continue

        mask = (df_imp[col].astype(str).str.strip() == 'Low_SNR')
        low_snr_sum = mask.sum()
        low_snr_ratio = low_snr_sum / len(df_imp)
        print(f"{col}: {low_snr_ratio:.2%} Low_SNR")

        if low_snr_ratio > dead_channel_threshold:                # MARTWY KANAŁ - oznaczamy cały kanał jako bad
            bad_channels.append(col)
            print(
                f" -> kanał oznaczony jako BAD "
                f"(Low_SNR = {low_snr_ratio:.2%})"
            )
            continue

        bad_times = df_imp.loc[mask, "Time"].to_numpy()           # KANAŁ W WIĘKSZOŚCI DOBRY - tworzymy adnotacje czasowe
        if len(bad_times) == 0:
            continue

        start = bad_times[0]                                     # znacznik początku obecnie rozważanego fragmentu
        prev = bad_times[0]                                     # znacznik poprzedniej rozważanej części fragmentu

        for current in bad_times[1:]:
            if current - prev > merge_gap:
                annotations.append(
                    dict(
                        onset=float(start),
                        duration=float(prev - start + 1),
                        description=f"BAD_{col}"
                    )
                )
                start = current
                prev = current
        # ostatni fragment
        annotations.append(
            dict(
                onset=float(start),
                duration=float(prev - start + 1),
                description=f"BAD_{col}"
            )
        )

    bads_in_edf = []                                            # Mapowanie nazw kanałów z CSV na nazwy w EDF
    for bad_ch in bad_channels:
        matched = [ch for ch in raw_data.ch_names if bad_ch in ch]
        bads_in_edf.extend(matched)

    raw_data.info["bads"] = bads_in_edf
    print(f"Updated MNE Raw object. Bad channels: {raw_data.info['bads']}")

    if len(annotations) > 0:                                    # Tworzenie adnotacji MNE
        annot = mne.Annotations(
            onset=[a["onset"] for a in annotations],
            duration=[a["duration"] for a in annotations],
            description=[a["description"] for a in annotations],
            orig_time=raw_data.annotations.orig_time
        )
        raw_data.set_annotations(
            raw_data.annotations + annot
        )
        print(
            f"Dodano {len(annotations)} "
            f"adnotacji związanych z Low_SNR."
        )

```

Rys. 7. Fragment kodu pipeline'u przygotowującego dane, odpowiedzialny za wykrywanie i oznaczanie kanałów o niskiej jakości.

Kanały, dla których udział próbek o niskiej jakości przekraczał ustalony próg (50%), uznawano za trwale uszkodzone i oznaczano jako złe (ang. *bad*). Były one pomijane w dalszych obliczeniach. W przypadku kanałów, które jedynie chwilowo traciły jakość, tworzono natomiast adnotacje czasowe obejmujące wyłącznie problematyczne fragmenty zapisu. Sąsiadujące ze sobą wadliwe odcinki łączono przy tym w jeden przedział, aby uniknąć nadmiernego rozdrobnienia adnotacji. Oznaczone w ten sposób fragmenty wykorzystano na późniejszym etapie do automatycznego odrzucania wadliwych prób.

4.6. Analiza składowych niezależnych (ICA)

Kolejnym krokiem czyszczenia sygnału było usunięcie artefaktów za pomocą analizy składowych niezależnych (ang. *Independent Component Analysis*, ICA). Metoda ta rozkłada zarejestrowany sygnał na zbiór składowych, z których część odpowiada aktywności mózgu, a część – zakłóceniom, takim jak mruganie, ruchy gałek ocznych czy napięcie mięśni. Usunięcie składowych będących artefaktami pozwala oczyścić sygnał przy zachowaniu właściwej aktywności.

```
#-----Analiza Komponentów Sygnału-----
def independent_component_anlysis():
    global raw_data

    ica = mne.preprocessing.ICA(
        n_components=0.99,
        method="fastica",
        random_state=42,
        max_iter="auto"
    )

    print("Dopasowywanie ICA...")
    ica.fit(raw_data)

    eog_inds, scores = ica.find_bads_eog(
        raw_data, # Wykrywanie komponentów związanych z mrugnięciami (EOG)
        ch_name=["X1:", "X2:"]
    )
    print(f"Znalezione komponenty EOG: {eog_inds}")

    ica.exclude.extend(eog_inds) # Oznaczenie komponentów do usunięcia
    try:
        muscle_inds, muscle_scores = ica.find_bads_muscle(
            raw_data # Wykrywanie artefaktów mięśniowych
        )
        print(f"Znalezione komponenty mięśniowe: {muscle_inds}")

        if len(muscle_inds) > 0:
            # UWAGA:
            # Na początku warto obejrzeć komponenty ręcznie
            # przed automatycznym usuwaniem.
            ica.exclude.extend(muscle_inds)
    except Exception as e:
        print("Nie udało się wykryć komponentów mięśniowych:")
        print(e)

    print(f"Usuwane komponenty ICA: {ica.exclude}") # Usunięcie wybranych komponentów
    raw_data = ica.apply(raw_data.copy())
    print("ICA zakończona.")
```

Rys. 8. Fragment kodu pipeline’u przygotowującego dane, odpowiedzialny za analizę składowych niezależnych (ICA).

Model ICA dopasowano do danych, przyjmując liczbę składowych odpowiadającą zachowaniu 99% wariancji sygnału odniesienia (EOG). Dodatkowo podjęto próbę automatycznego wykrycia składowych o charakterze mięśniowym. Wskazane składowe oznaczono jako niepożądane i odtworzono sygnał z ich pominięciem. W zastosowanym rozwiązaniu identyfikacja składowych przebiegała automatycznie; w docelowym postępowaniu wskazana byłaby dodatkowo ich wizualna weryfikacja.

4.7. Podział sygnału na epoki

Do dalszej analizy konieczne było wyodrębnienie z ciągłego zapisu fragmentów odpowiadających prezentacji poszczególnych obrazów.

```
#-----Tworzenie i wizualizacja Eventów-----
def create_epochs_from_ImageOn_events():
    global raw_data
    global epochs

    events = mne.find_events(raw_data, stim_channel="Trigger", initial_event=True, shortest_event=0.5, min_duration=0.001, consecutive=True)
    print(f"Liczba wszystkich eventów: {len(events)}")
    #-----Filtracja eventu IMAGE_ON-----
    imageOn_events = events[(events[:, 2] == 12)]
    #-----Porównanie Ilości znalezionych eventów z dziennikiem w CSV-----
    events_csv = pd.read_csv(f"assets/{CSV_EVENTY}.csv")
    image_events_csv = (events_csv.query("event_code == 12").reset_index(drop=True)) #przeszukanie csv za IMAGE_ON
    print(f"Liczba IMAGE_ON w CSV: {len(image_events_csv).csv}")

    assert len(imageOn_events) == len(image_events_csv), \
        "Liczba eventów IMAGE_ON w EEG i w CSV nie jest taka sama!"
    #-----Utworzenie metadanych dla epok-----
    metadata = image_events_csv[
        [ "participant_id", "series_id", "trial_id",
          "image_id", "image_category", "image_file" ]
    ].copy()

    event_id = {"IMAGE_ON": 12} # Definicja event_id
    #-----Utworzenie epok-----
    epochs = mne.Epochs(
        raw_data,          # t = 0      -> pojawienie się obrazka
        imageOn_events,    # t = -0.35  -> 350 ms przed obrazkiem
        event_id=event_id, # t = 0.5   -> 500 ms po rozpoczęciu wyświetlania
        tmin=-0.35,
        tmax=0.5,
        baseline=(-0.35, -0.01),
        metadata=metadata,
        preload=True,
        reject_by_annotation=True # odrzucenie epok z adnotacjami BAD_
    )
    usuniete = [idx for idx, log in enumerate(epochs.drop_log) if len(log) > 0]
    print(f"Lista usuniętych epok: {(usuniete)}")
```

Rys. 9. Fragment kodu pipeline'u przygotowującego dane, odpowiedzialny za podział sygnału na epoki.

W tym celu na podstawie kanału **Trigger** odczytano zdarzenia zarejestrowane podczas badania, a następnie wybrano spośród nich te oznaczone kodem odpowiadającym pojawieniu się obrazu (**IMAGE_ON**). Liczbę tak wyznaczonych zdarzeń porównano z liczbą obrazów zapisaną w pliku ze zdarzeniami; zgodność obu wartości stanowiła zabezpieczenie przed niespójnością danych, a w przypadku jej braku przetwarzanie było przerywane.

Dla każdego zdarzenia utworzono epokę, czyli wycinek sygnału obejmujący okno od 0,35 s przed pojawieniem się obrazu do 0,5 s po nim. Fragment poprzedzający bodziec wykorzystano jako linię bazową (ang. **baseline**), względem której korygowano sygnał, co pozwalało ograniczyć wpływ różnic w poziomie sygnału pomiędzy próbami. Do każdej epoki dołączono również informacje opisowe (metadane), takie jak numer serii, numer próby oraz kategoria i nazwa prezentowanego obrazu, dzięki czemu możliwe było powiązanie sygnału z konkretnym bodźcem. Na tym etapie odrzucono także epoki pokrywające się z wcześniej oznaczonymi wadliwymi fragmentami zapisu.

4.8. Analiza czasowo-częstotliwościowa

Przygotowane epoki poddano analizie czasowo-częstotliwościowej (ang. *time-frequency analysis*), która pozwala prześledzić, jak moc sygnału w poszczególnych pasmach częstotliwości zmienia się w czasie.

```
#-----Analiza w Time-Freq używając falek Morleta-----
def morlet_wavelet():
    global epochs
    global tfData_float32

    if(do_morlet):
        czestotliwosci = np.arange(4,40, 1)
        l_cykli = czestotliwosci/4
        tfa = epochs.compute_tfr(method="morlet", freqs=czestotliwosci, n_cycles=l_cykli,
                                decim=1, picks='eeg', return_itc=False, average=False)

        if reduct_tfa_using_baseline: #redukcja sygnału względem funkcji na w odniesieniu do sygnału przed wydarzeniem
            tfa.apply_baseline(baseline=(-0.3, -0.05), mode=chosen_reduction_method)

        if morlet_log_scale:
            #tfa.apply_log_transform()
            tfData = tfa.crop(tmin=0.0, tmax=0.5).get_data()
            tfData = 10 * np.log10(tfData + 1e-10)
            tfData_float32 = tfData.astype(np.float32)
        else:
            tfData = tfa.crop(tmin=0.0, tmax=0.5).get_data()
            tfData_float32 = tfData.astype(np.float32)

        if subtract_mean_baseline:
            srednia = np.mean(tfData_float32, axis=1, keepdims=True) #LEKKIE POLEPSZENIE WYNIKOW
            tfData_float32 = tfData_float32-srednia
    else:
        tfData = epochs.copy().crop(tmin=0.0, tmax=0.5).get_data(picks='eeg')
        tfData_float32 = tfData.astype(np.float32)
        tfData_float32 = np.expand_dims(tfData_float32, axis = -1)

    print(tfData_float32)
```

Rys. 10. Fragment kodu pipeline'u przygotowującego dane, odpowiedzialny za analizę czasowo-częstotliwościową epok.

Wykorzystano w tym celu transformatę falkową z zastosowaniem falki Morleta, wyznaczając reprezentację sygnału dla częstotliwości z zakresu od 4 do 40 Hz. Liczbę cykli falki uzależniono od częstotliwości, co stanowi kompromis pomiędzy rozdzielczością czasową a częstotliwościową. Istotne było przy tym zachowanie osobnej reprezentacji dla każdej epoki tak, aby każdy obraz po połączeniu z etykietą stanowił osobny przykład uczący.

Uzyskaną reprezentację ograniczono do okna od 0 do 0,5 s, odpowiadającego prezentacji obrazu. Wartości mocy przeliczono następnie na skalę logarytmiczną (decybele), co pozwoliło wyrównać znaczne różnice wartości pomiędzy pasami i uzyskać rozkład korzystniejszy z punktu widzenia uczenia sieci neuronowej. Dodatkowo przewidziano opcjonalne odniesienie sygnału do linii bazowej sprzed bodźca oraz odjęcie średniej wartości liczbowej pomiędzy kanałami, co w przeprowadzonych próbach nieznacznie poprawiało wyniki.

Niezależnie od powyższego przygotowano również wariant, w którym pomijano analizę czasowo-częstotliwościową, a modelowi przekazywano bezpośrednio przefiltrowany sygnał

z epok (dane surowe). Umożliwiało to porównanie skuteczności modelu na danych przekształconych oraz nieprzekształconych.

4.9. Zapis przygotowanych danych

Ostatnim etapem był zapis przygotowanych danych do plików w formacie `.npy`, umożliwiającym ich szybkie wczytanie na etapie uczenia modelu.

```
#-----Przygotowanie zmiennych do klasyfikacji-----
def save_to_file(nick):
    y_category = epochs.metadata["image_category"] # Kategoria obrazka
    y_image = epochs.metadata["image_id"] # Klasyfikacja według pliku

    print("Category y")
    print(y_category.head())
    print("Image y")
    print(y_image.head())
    #-----Zapis wyników do plików-----

    sub_mean_base_text = "AVG" #Ustalanie nazwy pliku na podstawie wybranych opcji
    reduct_text = ""
    if subtract_mean_baseline:
        sub_mean_base_text = "NoAVG"
    if reduct_tfa_using_baseline:
        reduct_text = chosen_reduction_method
    if not do_morlet:
        reduct_text = "RAW"

    np.save(f"data/{nick}_Dane32Przetworzone{sub_mean_base_text}{reduct_text}.npy", tfData_float32)
    np.save(f"data/{nick}_EtykietyDanych{sub_mean_base_text}{reduct_text}.npy", y_category)
    print("Utworzono Pliki")
```

Rys. 11. Fragment kodu pipeline'u przygotowującego dane, odpowiedzialny za zapis przygotowanych danych.

Osobno zapisywano tablicę z przetworzonym sygnałem oraz plik zawierający etykiety kategorii obrazów. W nazwach plików uwzględniono zastosowany wariant przetwarzania (m.in. dane przekształcone lub surowe), dzięki czemu poszczególne wersje zbioru danych nie nadpisywały się wzajemnie i mogły być ze sobą porównywane.

4.10. Wizualizacja danych

W celu jakościowej oceny przygotowanych danych przewidziano zestaw funkcji pomocniczych, generujących wizualizacje przetworzonego sygnału. Nie stanowiły one części właściwego potoku przekazywanego modelowi, lecz służyły jedynie do podglądu i weryfikacji poprawności kolejnych etapów. Każda z funkcji przedstawiała spektrogramy poszczególnych epok w postaci obrazów (map czasowo-częstotliwościowych) i zapisywała je jako animacje w formacie GIF, w katalogu wskazanym zmienną `visualization_catalogue`.

Pierwsza z funkcji przedstawiała spektrogramy pojedynczego kanału dla kolejnych epok w porządku chronologicznym, czyli w kolejności ich rejestracji podczas badania. Pozwalało to prześledzić, jak zmienia się sygnał na wybranej elektrodzie w trakcie eksperymentu.

```

# Pojedynczy obrazek dla jednej epoki dla jednego kanału
def generate_single_epoche_image(): 1 usage 2 Szonny
    global tfData_float32
    global image_epoch
    global image_channel

    jedenWynik = tfData_float32[image_epoch, image_channel, :, :]

    fig, ax = plt.subplots(figsize=(8, 6))
    im = ax.imshow(jedenWynik, cmap="jet", aspect="auto", origin="lower")

    ax.set_title(f"Epoka {image_epoch}, Kanał {image_channel}")
    ax.set_xlabel("Punkty czasu")
    ax.set_ylabel("Indeksy częstotliwości")
    fig.colorbar(im, label="Moc sygnału")

    plt.savefig(f"{visualization_catalogue}/{NICK_BADANE60}_spektrogram_epoka_{image_epoch}.png", dpi=100, bbox_inches="tight")
#
    plt.show()
    plt.close()
    print("Obrazek PNG został zapisany!")
# GIF dla jednego kanału z różnych epok po kolei

```

Rys. 12. Fragment kodu pipeline'u przygotowującego dane, przedstawiający spektrogramy pojedynczego kanału dla kolejnych epok w porządku chronologicznym.

Druga funkcja porządkowała epoki nie według czasu, lecz według prezentowanego obrazu i jego kategorii. Epoki sortowano na podstawie identyfikatora obrazu, a następnie wyznaczano granice pomiędzy kolejnymi kategoriami, dzięki czemu możliwe było wyświetlenie spektrogramów odpowiadających jednej, wybranej kategorii (określonej zmienną `chosen_category`). Umożliwiało to porównanie sygnału rejestrowanego dla obrazów należących do tej samej klasy.

```

# GIF dla jednego kanału z epok posortowanych według obrazka
def generate_single_channel_gif_in_image_order(): 1 usage 3 Szonny
    global current_category
    y_image = epochs.metadata["image_id"] # Klasyfikacja według pliku
    y_category = epochs.metadata["image_category"] # Kategoria obrazka

    nr_kanalů = image_channel
    frames = []

    y_cat_np = y_category.to_numpy()
    y_img_np = y_image.to_numpy()
    sort_indek = np.argsort(y_img_np)

    warunek_zm = y_cat_np[sort_indek[:-1]] != y_cat_np[sort_indek[1:]]
    p_zmiany = np.where(warunek_zm)[0] + 1
    granice = [0] + list(p_zmiany) + [len(sort_indek)]

    for nrKlatki, nr_epoki in enumerate(sort_indek[granice[current_category]:granice[current_category+1]]):
        fig, ax = plt.subplots(figsize=(8, 6))

        single_spectrogram = tfData_float32[nr_epoki, nr_kanalů, :, :]

        obecnyobraz = y_img_np[nr_epoki]

        ax.imshow(single_spectrogram, cmap="jet", aspect="auto", origin="lower")
        ax.set_title(f"Kanał: {raw_data.ch_names[nr_kanalů]} "
                    f"Epoka: {nr_epoki} Obraz {obecnyobraz}")

        buf = io.BytesIO()
        plt.savefig(buf, format="png", bbox_inches="tight")
        buf.seek(0)

        frames.append(io.imread(buf))
        plt.close(fig)

    # duration=1000 sekundna na klatke
    io.imwrite(uri=f"{visualization_catalogue}/{NICK_BADANEGO}_animacja_epok.gif", frames, duration=400, loop=0)
    print("GIF utworzony!")
#GIF dla elektrod F3, F4, C3, C4 01, 02 dla posortowanych obrazków

```

Rys. 13. Fragment kodu pipeline'u przygotowującego dane, porządkujący epoki według prezentowanego obrazu i jego kategorii

Trzecia funkcja rozszerzała powyższe podejście na sześć elektrod jednocześnie, prezentowanych w układzie odpowiadającym ich położeniu na głowie. W jednym kadrze zestawiano elektrody z lewej (F3, C3, O1) oraz prawej (F4, C4, O2) półkuli, w tym najistotniejsze dla analizy elektrody potyliczne O1 i O2. Podobnie jak poprzednio, epoki porządkowano według kategorii obrazu, a wynik zapisywano jako pojedynczą animację.

```

#GIF dla elektrod F3, F4, C3, C4 01, 02 dla posortowanych obrazków
def generate_combo_GIF_image_order(): 1 usage & Szonny *
    global current_category
    y_category = epochs.metadata["image_category"] # Kategoria obrazka
    y_image = epochs.metadata["image_id"] # Klasyfikacja według pliku
    y_cat_np = y_category.to_numpy()
    y_img_np = y_image.to_numpy()
    sort_indek = np.argsort(y_img_np)
    frames = []
    kanalyL = [2,1,14]
    kanalyP = [4,5,15]
    warunek_zm = y_cat_np[sort_indek[:-1]] != y_cat_np[sort_indek[1:]]
    p_zmiany = np.where(warunek_zm)[0] + 1
    granice = [0] + list(p_zmiany) + [len(sort_indek)]

    for nrKlatki, nr_epoki in enumerate(sort_indek[granice[current_category]:granice[current_category+1]]):
        fig, ax = plt.subplots(nrows=3, ncols=2, figsize=(8, 6))

        obecnyobraz = y_img_np[nr_epoki]
        for idy, nr_kanal in enumerate(kanalyL):
            single_spectrogram = tfData_float32[nr_epoki, nr_kanal, :, :]
            ax[idy,0].imshow(single_spectrogram, cmap="jet", aspect="auto", origin="lower")
            ax[idy,0].set_title(f"Kanał: {raw_data.ch_names[nr_kanal]}")

        for idy, nr_kanal in enumerate(kanalyP):
            single_spectrogram = tfData_float32[nr_epoki, nr_kanal, :, :]
            ax[idy,1].imshow(single_spectrogram, cmap="jet", aspect="auto", origin="lower")
            ax[idy,1].set_title(f"Kanał: {raw_data.ch_names[nr_kanal]}")

        fig.suptitle(f"Obraz {obecnyobraz}", fontsize=10, fontweight='bold')
        ax[2,1].set_xlabel("Czas")
        ax[1,0].set_ylabel("Częstotliwość")
        buf = io.BytesIO()
        plt.savefig(buf, format="png", dpi=100)
        buf.seek(0)

        frames.append(iio.imread(buf))
        plt.close(fig)

    # duration=1000 sekund na klatke
    iio.imwrite(uri: f"{visualization_catalogue}/{NICK_BADANEGO}_animacja_epokKanalow_{now}.gif", frames, duration=600, loop=0)

```

Rys. 14. Fragment kodu pipeline'u przygotowującego dane, który prezentuje rozkład sygnału na elektrodach.

4.11. Parametryzacja potoku

Całość przetwarzania spięto w jedną funkcję sterującą, przyjmującą jako argumenty najważniejsze ustawienia potoku (m.in. wybór badanych, zastosowanie analizy czasowo-częstotliwościowej czy sposób normalizacji).

```

def generuj_pliki(lista_nr_badanych=[0], do_TFA=True, morlet_log=True, subtract_mean=False, reduct_tfa_baseline = False, red_method=2):
    global wybrani_badani, do_generate_single_image, do_generate_single_gif_chrono_order, do_generate_single_gif_image_order, do_generate_combo_gif
    global do_morlet, NICK_BADANEGO, CSV_EVENTY
    global morlet_log_scale
    global subtract_mean_baseline
    global reduct_tfa_using_baseline
    global chosen_reduction_method

    for nick in nicki_badanych:
        if not os.path.exists(f"assets/{nick}_raw.edf"):
            print(f"Nie znaleziono pliku EDF: {nick}_raw.edf")
        if not os.path.exists(f"assets/{nick}_imp.csv"):
            print(f"Nie znaleziono pliku CSV: {nick}_imp.csv")

    if not os.path.exists("data"):
        os.mkdir("data")

    # Wybór obecnych badanych
    wybrani_badani = lista_nr_badanych

    # Opcje edycji epok
    do_morlet = do_TFA # Czy dane po TFA czy surowe?
    morlet_log_scale = morlet_log
    subtract_mean_baseline = subtract_mean

    reduct_tfa_using_baseline = reduct_tfa_baseline
    chosen_reduction_method = reduction_methods[red_method]

    if not do_morlet:
        do_generate_single_image = False
        do_generate_single_gif_chrono_order = False
        do_generate_single_gif_image_order = False
        do_generate_combo_gif = False

    for i in wybrani_badani:
        NICK_BADANEGO = nicki_badanych[i]
        CSV_EVENTY = csv_eventow[i]

        load_data(NICK_BADANEGO)
        change_channel_names()
        change_channel_types()
        change_montage_and_reference()
        drop_unimportant_channels()
        filter_raw_data()
        delete_bad_channels()
        independent_component_anlysis()
        create_epochs_from_ImageOn_events()
        morlet_wavelet()
        if do_save_to_file:
            save_to_file(NICK_BADANEGO)
        if do_generate_single_image:
            generate_single_epoche_image()
        if do_generate_single_gif_chrono_order:
            generate_single_channel_gif_in_chrono_order()
        if do_generate_single_gif_image_order:
            generate_single_channel_gif_in_image_order()
        if do_generate_combo_gif:
            generate_combo_GIF_image_order()

    if __name__ == "__main__":
        generuj_pliki()

```

Rys. 15. Funkcja sterująca, dzięki której można wyzwać potok z różnymi parametrami.

Rozwiązanie to pozwalało wygodnie generować różne warianty zbioru danych bez konieczności ingerencji w kod, co ułatwiało systematyczne porównywanie wpływu poszczególnych ustawień na wyniki. W początkowej wersji projektu potok miał postać pojedynczego, liniowego skryptu; parametryzację wprowadzono na późniejszym etapie, gdy pojawiła się potrzeba prowadzenia wielu eksperymentów.

4.12. Mieszanie i podział danych

Pliki przygotowane dla poszczególnych badanych łączono następnie w jeden zbiór, który mieszano i dzielono na części wykorzystywane podczas uczenia.

```

#-----Wczytywanie danych-----
def load_data(nick):
    global raw_data

    raw_data = mne.io.read_raw_edf(f"assets/{nick}.raw.edf", preload=True, infer_types=True) # załaduj do pamięci, typ EEG do znanych kanałów
    print("\n#####\n")
    print(raw_data) #Informacje o kanałach i obiekcie
    print("\n#####\n")
    print(raw_data.info) #Informacje o kanałach i obiekcie
    print("\n#####\n")
    print(raw_data.describe()) #Informacje o kanałach i obiekcie
    print("\n#####\n")

```

Rys. 16. Fragment kodu pipeline’u przygotowującego dane, odpowiedzialny za wczytanie surowych danych do programu.

Podziału dokonano w sposób rozłączny względem uczestników: część ucząca obejmowała siedmiu badanych, a część testowa – dwóch pozostałych. Dzięki temu model oceniano na danych osób, których nagrania nie były wykorzystywane podczas treningu. Przewidziano również osobną funkcję przygotowującą zbiór walidacyjny.

Ze względu na znaczny rozmiar danych, przekraczający pojemność pamięci operacyjnej, mieszanie przeprowadzono dwuetapowo, bez wczytywania całości naraz. Najpierw wyznaczano losową kolejność wszystkich prób, a następnie dane rozdrabniano, zapisując je do plików tymczasowych zgodnie z tą kolejnością. W drugim etapie fragmenty te scalano z powrotem w kilku plikach wynikowych, dodatkowo tasując zawartość każdego z nich. Podział na wiele mniejszych plików ułatwiał późniejsze wczytywanie danych partiami.

Poprawność operacji weryfikowano, sprawdzając, czy ustalona próbka prób pochodzących z danych wejściowych – wraz z przypisanymi im etykietami – odnalazła się w plikach wynikowych. Pozwalało to wykryć ewentualne zagubienie lub niezgodność danych powstałe podczas mieszania.

Na koniec dane poddawano normalizacji. Dla każdego kanału wyznaczano średnią oraz odchylenie standardowe, a następnie standaryzowano wartości, odejmując średnią i dzieląc przez odchylenie standardowe. Wyznaczone parametry zapisywano osobno, aby możliwe było ich późniejsze wykorzystanie. Ostatnim krokiem była zmiana kolejności wymiarów tablicy na układ oczekiwany przez model, z kanałami umieszczonymi jako ostatni wymiar.

4.13. Uruchomienie potoku i generowanie zbioru danych

Całościowe wygenerowanie zbioru danych realizowano za pomocą osobnego skryptu sterującego, który spinał dwa etapy: przygotowanie danych (opisany wyżej potok, wywoływany funkcją `generuj_pliki`) oraz ich późniejsze wymieszanie i przygotowanie postaci wynikowej (funkcja `generuj_wynikowe` z osobnego modułu). Ustawienia wybrane w skrypcie (m.in. zastosowanie analizy czasowo-częstotliwościowej oraz metoda normalizacji) przekazywano do obu etapów, dzięki czemu pozostawały one ze sobą spójne. Takie rozwiązanie pozwalało uruchomić cały proces – od surowego zapisu EEG aż po gotowy zbiór – jednym poleceniem.

Skrypt przewidywał dwa tryby działania, przełączane flagą. W trybie podstawowym generowano pojedynczy wariant zbioru danych: dla wszystkich dziewięciu badanych wykonywano przygotowanie danych z ustalonymi parametrami, a następnie mieszano wynik. Tryb ten wykorzystywano podczas bieżącej pracy, gdy potrzebna była jedna konkretna wersja danych.

Drugi tryb służył do hurtowego wygenerowania wielu wariantów zbioru naraz. Kolejno wywoływano w nim przygotowanie i mieszanie danych dla różnych ustawień normalizacji – m.in. z odejmowaniem średniej pomiędzy kanałami oraz dla poszczególnych metod odniesienia do linii bazowej (*ratio*, *logratio*, *zlogratio*, *mean* oraz *zscore*). Pozwalało to przygotować komplet danych do porównania wpływu poszczególnych metod na wyniki, przy czym – ze względu na liczbę wariantów – powstający zbiór osiągał znaczny rozmiar (rzędu ponad 130 GB).

5. Klasyfikacja obrazów

Klasyfikację obrazów na podstawie sygnału EEG zrealizowano, porównując trzy architektury sieci neuronowych oraz dwie reprezentacje sygnału. Pierwsze podejście wykorzystywało klasyczną sieć konwolucyjną (CNN) działającą na reprezentacji czasowo-częstotliwościowej (spektrogramach), natomiast dwa pozostałe – sieci EEGNet oraz EEG-Inception – operowały bezpośrednio na surowym sygnale w dziedzinie czasu. Zadaniem każdego modelu było przypisanie próby do jednej z jedenastu kategorii obrazów. W toku prac szczególnie istotny okazał się sposób podziału danych na zbiór uczący i testowy, któremu poświęcono osobny podrozdział.

5.1. Sieć konwolucyjna (CNN), z wykorzystaniem spektrogramów

Pierwszy model stanowiła klasyczna sieć konwolucyjna, traktująca reprezentację czasowo-częstotliwościową pojedynczej próby jak obraz o 21 kanałach, odpowiadających 21 elektrodą EEG. Każdy spektrogram posiadał wysokość 36, ponieważ był wykonany dla częstotliwości 4-40 Hz, oraz 301 wartości szerokości odpowiadającym pół sekundy przy częstotliwości próbkowania 600 Hz.

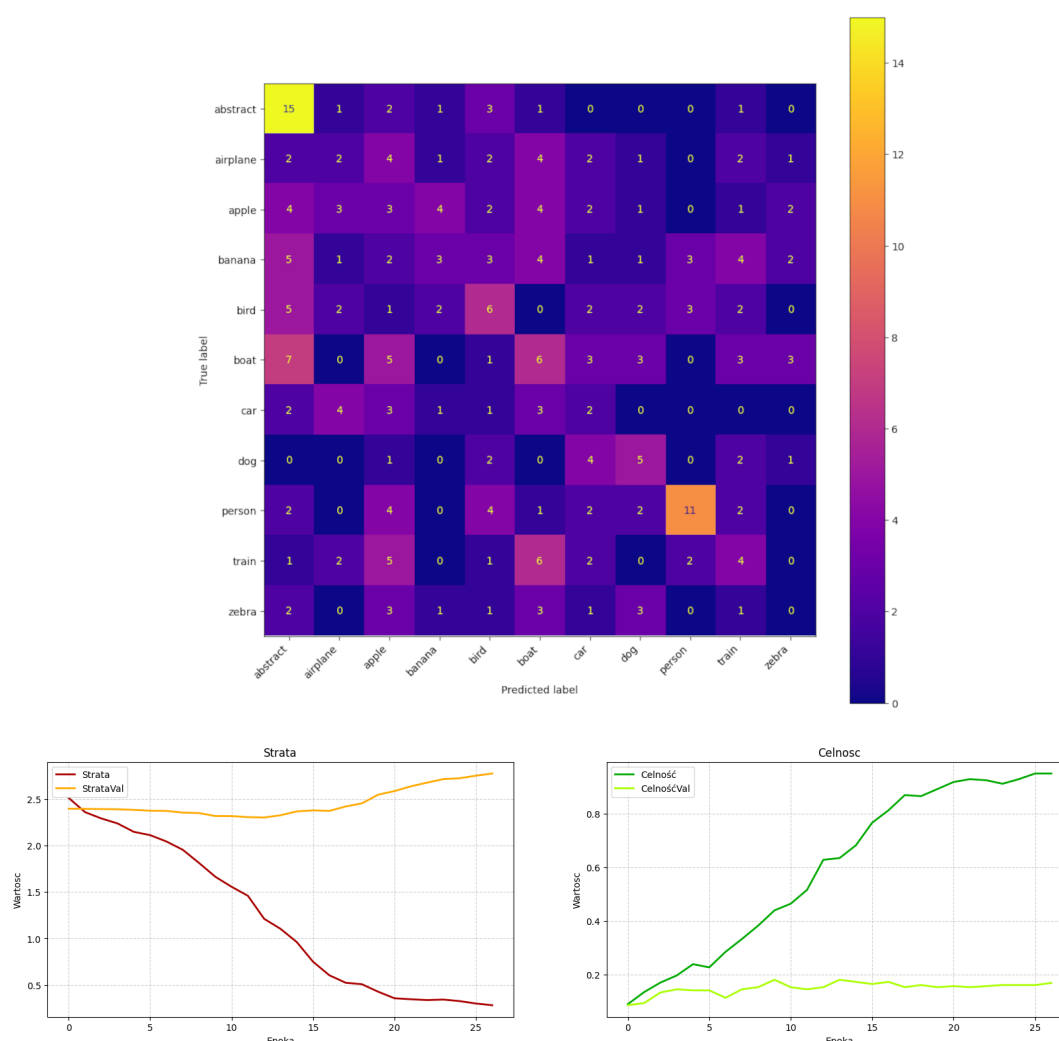
Sieć zbudowano z kilku bloków konwolucyjnych ze splotami rozciągniętymi wzdłuż osi czasu oraz częstotliwości (*conv2d*) i splotami typu *depthwise*, zakończonych częścią gęstą z warstwą *softmax* (*dense*). W celu zapobiegania przed przeuczeniem sieci, wprowadzono ograniczenia szybkości uczenia. Zastosowano obniżony *learning-rate* (0.0001), normalizację wsadową po każdej warstwie konwolucyjnej (ang. *batch normalization*) oraz *dropout* wysokich wartościach wyłączający losowo części sieci podczas trenowania. Po nich wykorzystano operacje łączące (ang. *max pooling*) oraz aktywację ReLU. Model uczono optymalizatorem Adam z funkcją straty *sparse_categorical_crossentropy*, wspomagany wczesnym zatrzymaniem oraz automatyczną redukcją współczynnika uczenia.

Model: "sequential_20"		
Layer (type)	Output Shape	Param #
conv2d_40 (Conv2D)	(None, 36, 301, 32)	6,752
batch_normalization_100 (BatchNormalization)	(None, 36, 301, 32)	128
activation_100 (Activation)	(None, 36, 301, 32)	0
max_pooling2d_60 (MaxPooling2D)	(None, 18, 150, 32)	0
conv2d_41 (Conv2D)	(None, 18, 150, 64)	6,208
batch_normalization_101 (BatchNormalization)	(None, 18, 150, 64)	256
activation_101 (Activation)	(None, 18, 150, 64)	0
max_pooling2d_61 (MaxPooling2D)	(None, 9, 50, 64)	0
dropout_40 (Dropout)	(None, 9, 50, 64)	0
depthwise_conv2d_40 (DepthwiseConv2D)	(None, 3, 17, 128)	524,416
batch_normalization_102 (BatchNormalization)	(None, 3, 17, 128)	512
activation_102 (Activation)	(None, 3, 17, 128)	0
depthwise_conv2d_41 (DepthwiseConv2D)	(None, 2, 9, 128)	524,416
batch_normalization_103 (BatchNormalization)	(None, 2, 9, 128)	512
activation_103 (Activation)	(None, 2, 9, 128)	0
max_pooling2d_62 (MaxPooling2D)	(None, 1, 4, 128)	0
flatten_20 (Flatten)	(None, 512)	0
dense_40 (Dense)	(None, 64)	32,832
batch_normalization_104 (BatchNormalization)	(None, 64)	256
activation_104 (Activation)	(None, 64)	0
dropout_41 (Dropout)	(None, 64)	0
dense_41 (Dense)	(None, 11)	715
Total params: 1,097,003 (4.18 MB)		
Trainable params: 1,096,171 (4.18 MB)		
Non-trainable params: 832 (3.25 KB)		

Rys. 17. Podsumowanie, ukazujące wszystkie warstwy utworzonej na potrzeby projektu sieci CNN.

W trakcie uczenia model wykazywał wyraźną skłonność do nadmiernego dopasowania (ang. *overfitting*) – osiągał wysoką celność na zbiorze uczącym przy znacznie niższej na testowym – dlatego zdecydowano się na tak drastyczne zmniejszenie współczynnika uczenia i podniesienie współczynnika *dropout*.

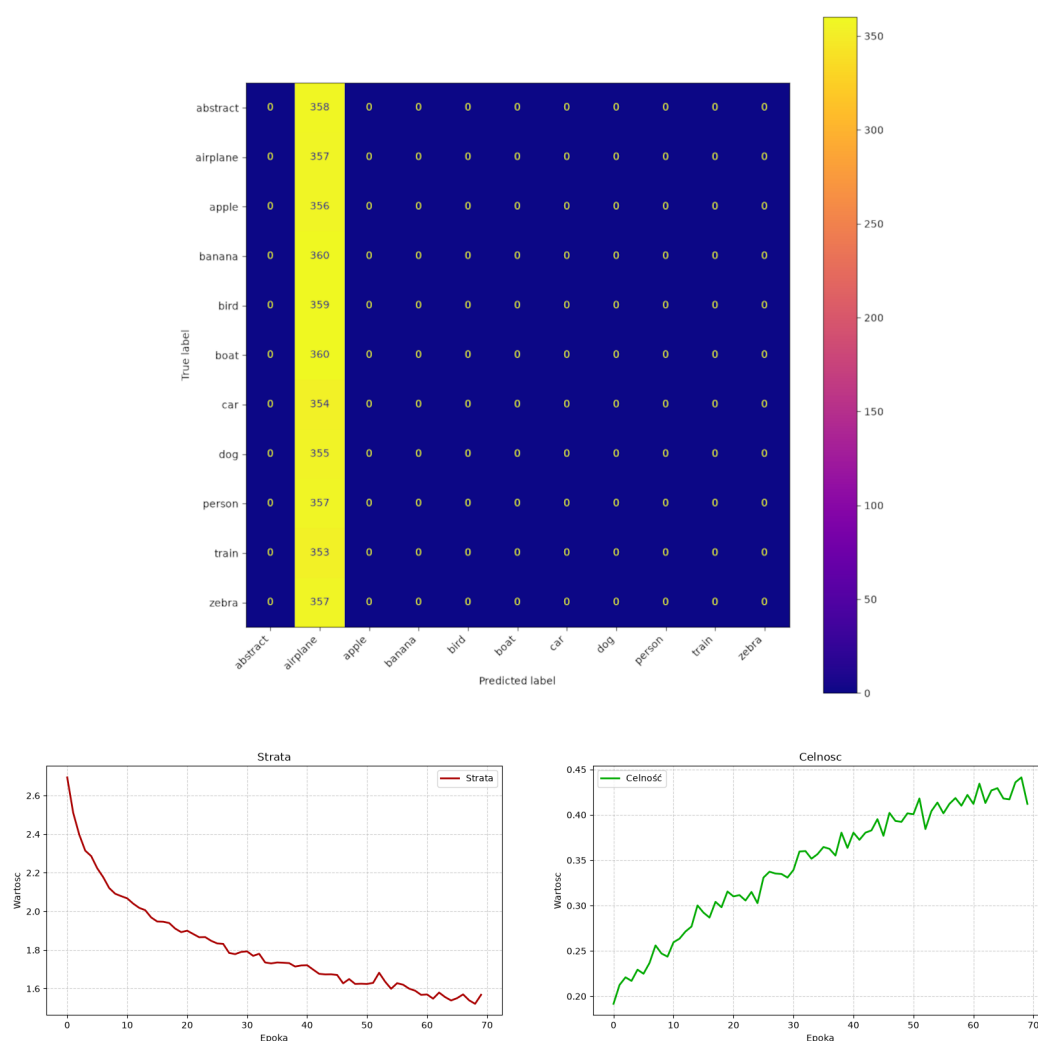
Przy testach dla pojedynczego pacjenta sieć wykazywała chęć do uczenia się, pomimo okrojonego zbioru danych potrafiła osiągnąć około 13% celności dla 11 kategorii. Istotnym usprawnieniem okazało się usuwanie średniej wartości pikseli między kanałami poszczególnych obrazków oraz przejście na logarytmiczną skalę wartości, miało to na celu wyrównanie duże różnice wartości pomiędzy pasmami, gdyż w niektórych reakcje były silniejsze, niż w pozostałych. Dzięki tym zmianą sieć ucząca się na spektrogramach podniosła swoją celność do 22.53% na zbiorze danych dla pojedynczego badanego.



Rys. 18. Wyniki dla najlepszego sieci, wyszkolonej na danych z pojedynczego pacjenta.

Pierwsze próby po utworzeniu skryptu mieszającego i ładującego, opierały się na idei zbioru treningowego, wykonanego z danych pochodzących od pierwszych 7 pacjentów i niezależnego zbioru testowego w którego skład wchodziły przetworzone dane od dwóch pozostałych pacjentów. Model był testowany na takich paczkach danych trzykrotnie, przechodząc podczas treningu po każdej paczce w tej samej kolejności. Takie podejście okazało się przynosić słabe wyniki. Pomimo raportowanych podczas uczenia stabilnych spadku straty i w zroście celności do nawet 40%, na danych testowych model wykazywał że nie potrafi zareagować na dane pochodzące od pacjentów nieuwzględnionych w zbiorze treningowym.

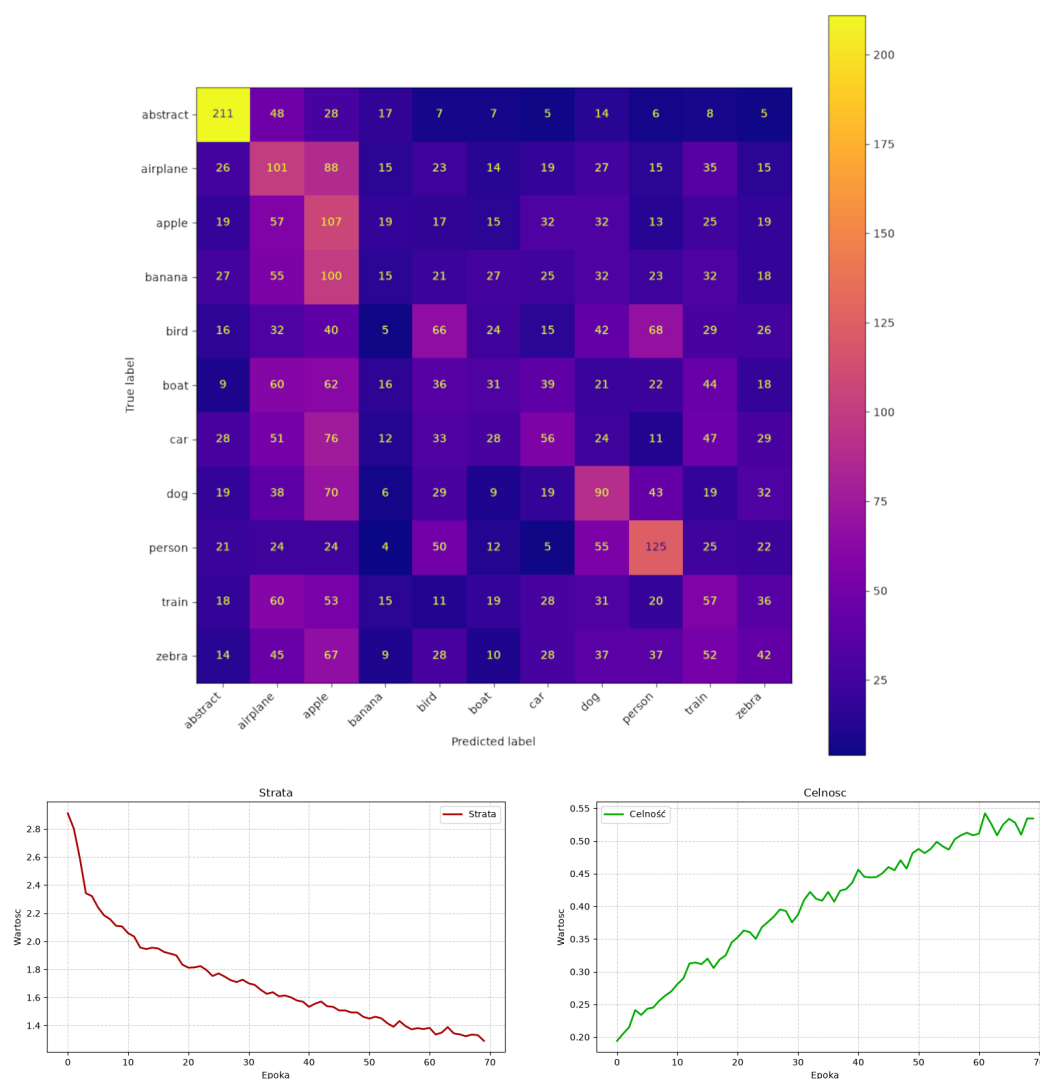
Dla wszystkich danych dostępnych spektrogramów, model potrafił osiągnąć 9,09% celności, czyli strzelał losowo jedną klasę.



Rys. 19. Wyniki dla najlepszej sieci, wyszkolonej na danych z wielu pacjentów.

Przy zniesieniu podziału pacjentów na treningowych i testowych, model znów zaczął osiągać wyniki podobne do które można było zaobserwować dla pojedynczego pacjenta. Model osiągnął 22,31% celności.

Późniejsze próby wzbogacenia redukcji wartości o średnią z 0,3s przed wyświetleniem obrazu nie przyniosły żadnej poprawy, jedynie redukując celność do 9.09% powodując ślepe zgadywanie klasy abstrakcyjnych obiektów.



Rys. 20. Wyniki dla najlepszej sieci, wyszkolonej na danych z wielu pacjentów pomieszanych między zbiór treningowy i testowy.

5.2. Sieć EEGNet

Drugą metodą była sieć EEGNet – kompaktowa architektura konwolucyjna zaprojektowana specjalnie dla sygnałów EEG, operująca bezpośrednio na surowym (przefiltrowanym) sygnale w postaci: kanały na próbki czasowe. Składa się ze splotu czasowego (uczącego się filtrów odpowiadających pasmom częstotliwości), splotu przestrzennego typu *depthwise* po kanałach oraz splotu separowalnego, z aktywacją ELU i ograniczeniami normy wag (ang. *max-norm*). Ze względu na niewielki rozmiar danych surowych model uczono w jednym przebiegu, z wydzieleniem części walidacyjnej.

Aby dopasować sygnał do założeń sieci, poddano go decymacji (podpróbkowaniu) – dane były odfiltrowane do 40 Hz, lecz próbkowane znacznie gęściej, więc zmniejszenie liczby próbek usunęło nadmiarową redundancję. W przeprowadzonych próbach model osiągał zbliżoną celność na zbiorze uczącym i walidacyjnym (rzędu 20–22%), nie potrafiąc dopasować się do danych treningowych powyżej tego poziomu. Wskazywało to na niedouczenie wynikające nie z ograniczeń samego modelu, lecz z ilości informacji zawartej w pojedynczej próbie surowego sygnału – zwiększanie pojemności sieci nie poprawiało wyniku.

5.3. Sieć EEG-Inception

Trzecią metodą była sieć EEG-Inception, również działająca na surowym sygnale, lecz wykorzystująca moduły typu *inception* – równoległe sploty czasowe o różnych długościach jądra, łączone ze sobą. Dzięki temu sieć analizuje sygnał w wielu skalach czasowych jednocześnie, wychytując zarówno krótsze, jak i dłuższe wzorce. Po splotach czasowych, podobnie jak w EEGNet, następuje spłot przestrzenny po kanałach. Architektura ta osiągnęła nieco lepsze wyniki niż EEGNet, pozostając jednak w tym samym, umiarkowanym zakresie skuteczności.

5.4. Podział danych i jego wpływ na wyniki

W trakcie prac kluczowe okazało się, w jaki sposób dane dzielone są na zbiór uczący i testowy. Rozważono dwa warianty. W podejściu *międzyosobowym* zbiór testowy tworzyli badani całkowicie nieobecni podczas treningu, co odpowiada pytaniu o zdolność modelu do generalizacji na nowe osoby. W podejściu *mieszanym* próby wszystkich badanych łączono, tasowano i dzielono losowo – zbiór testowy zawierał wówczas nowe próby osób znanych z treningu.

Różnica w wynikach okazała się bardzo duża. Przy podziale mieszanym modele osiągały celność wyraźnie powyżej poziomu losowego (13-22% dla jedenastu klas), natomiast przy podziale międzyosobowym spadały niemal do tego poziomu (ok. 9,26% dla jedenastu klas), przewidując dla danych testowych praktycznie jedną klasę. Oznacza to, że sieci uczyły się wzorców w dużej mierze specyficznych dla poszczególnych osób. Tę samą obserwację potwierdziła analiza spektrogramów: w obrębie jednego badanego istnieją powtarzalne zależności, jednak dwie różne osoby reagują na ten sam obraz odmiennie w dziedzinie czasu

i częstotliwości. Silna zmienność międzyosobowa sygnału EEG stanowiła zatem główne ograniczenie skuteczności klasyfikacji, przy uogólnianiu danych między pacjentów.

5.5. Wyniki i porównanie metod

Wyniki uzyskane przy podziale mieszanym zestawiono w tabeli 1. Wszystkie trzy modele osiągnęły zbliżony, umiarkowany poziom skuteczności – rzędu 20–24% wobec 9% poziomu losowego. Najlepiej wypadła sieć EEG-Inception, nieznacznie wyprzedzając model CNN oraz EEGNet. Model CNN, mimo najwyższej celności na zbiorze uczącym, wyraźnie się przeuczał, co ograniczało jego skuteczność na danych testowych. Przy podziale międzyosobowym skuteczność wszystkich modeli spadała do okolic poziomu losowego (dla modelu CNN odnotowano ok. 11%).

Model	Reprezentacja	Celność (test)
CNN	spektrogramy	ok. 22,5%
EEGNet	sygnał surowy	ok. 20%
EEG-Inception	sygnał surowy	ok. 23,5%

Tabela 1: Skuteczność klasyfikacji przy podziale mieszanym (poziom losowy ok. 9%).

We wszystkich przypadkach najlepiej rozpoznawaną kategorią był *abstract* – prosta grafika geometryczna, wizualnie skrajnie odmienna od zdjęć, wywołująca wyraźnie inną odpowiedź kory wzrokowej. Macierze pomyłek pokazywały ponadto tendencję modeli do częstego przewidywania kilku kategorii (m.in. samochodu, niekiedy jabłka), co przy słabym sygnale odpowiada dryfowi w stronę klas dominujących.

Podsumowując, dekodowanie kategorii oglądanego obrazu z pojedynczej próby EEG okazało się możliwe jedynie w ograniczonym zakresie. Sygnał niesie informację wystarczającą do odróżnienia wyraźnie odmiennych bodźców, jednak dokładne rozróżnienie jedenastu kategorii – zwłaszcza pomiędzy różnymi osobami – przekracza możliwości metody na tym poziomie. Reprezentacja czasowo-częstotliwościowa (CNN) oraz surowy sygnał (EEGNet, EEG-Inception) dały porównywalne rezultaty, co sugeruje, że wąskim gardłem nie jest architektura sieci, lecz ilość informacji o obrazie zawarta w samym sygnale.

6. Rekonstrukcja obrazów

Drugim celem projektu była próba rekonstrukcji obrazów na podstawie sygnału EEG, czyli odtworzenia informacji o obrazie oglądanym przez badanego wyłącznie na podstawie zarejestrowanej aktywności mózgowej. Zadanie to jest bardziej wymagające niż klasyfikacja, ponieważ wymaga odtworzenia znacznie bogatszej informacji niż jedynie przynależność obrazu do określonej kategorii. W ramach projektu zastosowano dwa komplementarne podejścia: bezpośrednią rekonstrukcję pikselową z wykorzystaniem autoenkodera oraz identyfikację obrazu spośród zbioru kandydatów (ang. *retrieval*). W obu przypadkach

wykorzystano surowy sygnał EEG oraz enkoder oparty na architekturze zgodnej z siecią EEGNet.

6.1. Rekonstrukcja pikselowa (autoenkoder)

Pierwsze podejście do rekonstrukcji oparto na autoenkoderze, czyli sieci neuronowej uczącej się odwzorowania danych wejściowych na odpowiadające im dane wyjściowe za pośrednictwem reprezentacji pośredniej. Autoenkoder składa się z dwóch współpracujących części. Enkoder przetwarza sygnał EEG i przekształca go w zwężony wektor cech (ang. *latent vector*), reprezentujący informację wyodrębnioną z sygnału. Dekoder wykorzystuje następnie tę reprezentację do wygenerowania obrazu. W procesie uczenia parametry obu części są dostosowywane w taki sposób, aby obraz odtworzony przez dekodera był możliwie zbliżony do obrazu rzeczywistego prezentowanego badanemu.

Enkoder zbudowano na bazie sieci EEGNet wykorzystanej wcześniej w zadaniu klasyfikacji, jednak usunięto z niej końcową warstwę klasyfikującą (**softmax**). Zamiast rozkładu prawdopodobieństwa po kategoriach enkoder zwraca wektor cech o ustalonej długości, wynoszącej w projekcie 128 wartości. Pozwala to wykorzystać wcześniej zastosowaną architekturę jako moduł ekstrakcji reprezentacji sygnału EEG. Dekoder rekonstruuje obraz etapami, stopniowo zwiększając jego rozdzielczość. W pierwszym kroku wektor cech jest przekształcany do niewielkiej mapy cech o rozmiarze 8×8 z wieloma kanałami. Następnie kolejne warstwy splotów transponowanych (ang. *transposed convolution*) zwiększają jej rozmiar, aż do uzyskania docelowej rozdzielczości 64×64 piksele. Ostatnia warstwa generuje trzy kanały odpowiadające składowym barwnym RGB, z wartościami z zakresu $[0, 1]$, tworząc w ten sposób wynikowy obraz. Kod obu części przedstawiono na rys. 24.

```

# ----- MODEL -----
def buduj_enkoder(Chans, Samples, latent_dim):
    """Korpus EEGNet zakończony wektorem cech (bez softmax)."""
    inp = Input((Chans, Samples, 1))
    x = Conv2D(16, (1, min(64, Samples)), padding="same", use_bias=False)(inp)
    x = BatchNormalization()(x)
    x = DepthwiseConv2D((Chans, 1), use_bias=False, depth_multiplier=2,
                        depthwise_constraint=max_norm(1.0))(x)
    x = BatchNormalization()(x); x = Activation("elu")(x)
    x = AveragePooling2D((1, 4))(x); x = Dropout(DROPOUT)(x)
    x = SeparableConv2D(32, (1, 16), padding="same", use_bias=False)(x)
    x = BatchNormalization()(x); x = Activation("elu")(x)
    x = AveragePooling2D((1, 8))(x); x = Dropout(DROPOUT)(x)
    x = Flatten()(x)
    latent = Dense(latent_dim, activation="elu", name="latent")(x)
    return Model(inp, latent, name="enkoder")

def buduj_dekoder(latent_dim, img_size=64, base=8):
    """Rozwija wektor cech do obrazu RGB przez sploty transponowane."""
    inp = Input((latent_dim,))
    x = Dense(base * base * 64, activation="elu")(inp)
    x = Reshape((base, base, 64))(x)
    n_up = int(np.log2(img_size // base)) # ile podwojen rozmiaru (8 -> img_size)
    filtry = 64
    for _ in range(n_up):
        filtry = max(filtry // 2, 16)
        x = Conv2DTranspose(filtry, 3, strides=2, padding="same", activation="elu")(x)
    out = Conv2D(3, 3, padding="same", activation="sigmoid")(x) # [0,1], IMG_SIZE^2 x 3
    return Model(inp, out, name="dekoder")

```

Rys. 21. Architektura enkodera (korpus EEGNet zakończony wektorem cech) oraz dekodera (sploty transponowane rozwijające wektor cech do obrazu).

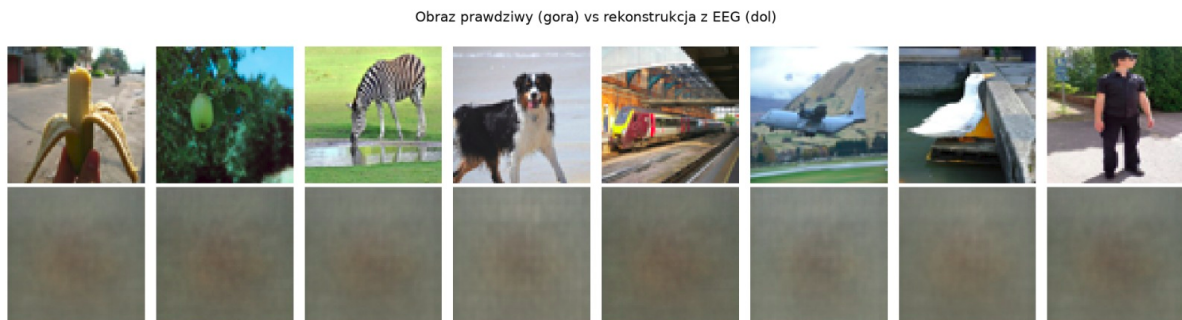
Obie części połączono w jeden model, w którym sygnał EEG jest kolejno przetwarzany przez enkoder i dekoder (rys. 26). Dla każdej próby jako docelowy wynik uczenia wykorzystywano rzeczywisty obraz prezentowany badanemu podczas rejestracji sygnału. Obrazy wczytywano z plików i przeskalowywano do rozdzielczości 64×64 piksele. Model był uczony z wykorzystaniem błędu średniokwadratowego (ang. *mean squared error*, MSE), obliczanego pomiędzy obrazem odtworzonym a obrazem docelowym. Funkcja ta określa średnią wartość kwadratów różnic pomiędzy odpowiadającymi sobie wartościami pikseli. Im niższa wartość MSE, tym mniejsza jest średnia różnica pomiędzy rekonstrukcją a obrazem docelowym.

```
def buduj_autoenkoder(Chans, Samples):
    enkoder = buduj_enkoder(Chans, Samples, LATENT_DIM)
    dekoderek = buduj_dekoder(LATENT_DIM, IMG_SIZE)
    wejscie = Input((Chans, Samples, 1))
    wyjscie = dekoderek(enkoder(wejscie))
    model = Model(wejscie, wyjscie, name="autoenkoder_eeg2obraz")
    model.compile(optimizer=Adam(LEARNING_RATE), loss="mse")
    model.summary()
    return model
```

Rys. 22. Złożenie autoenkodera z enkodera i dekodera oraz kompilacja z funkcją straty MSE (regresja do pikseli).

Uzyskane rekonstrukcje okazały się jednak bardzo podobne dla różnych prób. Niezależnie od podanego sygnału EEG model generował obrazy o zbliżonej, rozmytej strukturze i kolorystyce (rys. 26). Takie zachowanie jest charakterystyczne dla rekonstrukcji optymalizowanej z wykorzystaniem funkcji straty MSE w sytuacji, gdy na podstawie danych wejściowych nie można jednoznacznie określić docelowego obrazu. W takim przypadku minimalizacja średniego błędu może prowadzić do generowania obrazu stanowiącego przybliżenie średniej charakterystyki obrazów występujących w zbiorze, zamiast do odtworzenia konkretnego obrazu. W rezultacie uzyskane rekonstrukcje nie zachowywały wystarczającej ilości informacji pozwalającej na wierne odtworzenie pojedynczego obrazu.

Wynik ten wskazał na ograniczenia bezpośredniej rekonstrukcji pikselowej w rozpatrywanym zadaniu i stał się motywacją do zastosowania drugiego podejścia, opartego na identyfikacji obrazu spośród zadanego zbioru kandydatów.



Rys. 23. Rekonstrukcja pikselowa: obraz rzeczywisty (góra) oraz obraz odtworzony na podstawie sygnału EEG (dół). Rekonstrukcje uzyskane dla różnych prób wykazują podobną strukturę, co wskazuje na tendencję modelu do generowania uśrednionej reprezentacji obrazów ze zbioru.

6.2. Identyfikacja obrazów (retrieval)

Drugie podejście do rekonstrukcji omijało konieczność bezpośredniego generowania pikseli, a tym samym ograniczenie związane z tendencją modelu do generowania uśrednionej reprezentacji obrazów, opisane w poprzedniej sekcji. Zamiast odtwarzać obraz w przestrzeni pikseli, uczono sieć identyfikowania obrazu, który badany oglądał, poprzez odwzorowanie sygnału EEG do wspólnej przestrzeni cech obrazów.

Punktem wyjścia było opisanie każdego z 220 obrazów za pomocą zwężłego wektora liczb (ang. *embedding*). Wektory te wyznaczono przy użyciu pretrenowanej sieci rozpoznającej obrazy (MobileNetV2), która została wcześniej nauczona na milionach obrazów. Dzięki temu obrazy o podobnej charakterystyce wizualnej i semantycznej otrzymują zbliżone reprezentacje wektorowe, natomiast obrazy istotnie różniące się od siebie zajmują w tej przestrzeni bardziej odległe położenia. W ten sposób powstała wspólna przestrzeń reprezentacji obrazów, w której każdemu obrazowi odpowiada określony punkt. Wszystkie wektory znormalizowano do jednostkowej długości, co ułatwia ich późniejsze porównywanie. Sposób wyznaczania embeddingów przedstawiono na rys. 27.

```
def osadz_obrazy(unikalne_pliki):
    """Zwraca (M, D) embeddingow obrazow (znormalizowane L2)."""
    if TRYB_OSADZEN == "pretrenowany":
        from tensorflow.keras.applications import MobileNetV2
        from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
        obrazy = np.stack([wczytaj_obraz(f, IMG_SIZE) for f in unikalne_pliki])
        baza = MobileNetV2(include_top=False, weights="imagenet", pooling="avg",
                           input_shape=(IMG_SIZE, IMG_SIZE, 3))
        emb = baza.predict(preprocess_input(obrazy * 255.0), batch_size=32, verbose=0)
    else: # "piksele"
        maly = np.stack([wczytaj_obraz(f, PIXEL_SIZE) for f in unikalne_pliki])
        emb = maly.reshape(len(unikalne_pliki), -1)
    emb = emb / (np.linalg.norm(emb, axis=1, keepdims=True) + 1e-8)
    print(f"Embedding obrazow: {emb.shape} (tryb: {TRYB_OSADZEN})")
    return emb.astype(np.float32)
```

Rys. 24. Wyznaczanie embeddingów obrazów pretrenowaną siecią MobileNetV2 oraz ich normalizacja do jednostkowej długości.

Następnie enkoder EEG uczono odwzorowywać sygnał do punktu w tej samej przestrzeni reprezentacji, możliwie bliskiego obrazowi, który badany rzeczywiście oglądał. Enkoder oparto na architekturze EEGNet, a jego wyjściem był wektor cech o długości zgodnej z embeddingiem obrazu, również znormalizowany do jednostkowej długości. Jako funkcję straty wykorzystano podobieństwo kosinusowe (ang. *cosine similarity*), które określa stopień zgodności kierunków dwóch wektorów. Dla znormalizowanych wektorów wartość 1 oznacza identyczny kierunek, natomiast wartość 0 oznacza ich ortogonalność. Uczenie polegało zatem na dopasowaniu reprezentacji uzyskiwanej z sygnału EEG do reprezentacji właściwego obrazu (rys. 28).

```

# ----- ENKODER EEG -----
def buduj_ekoder_eeg(Chans, Samples, embed_dim):
    inp = Input((Chans, Samples, 1))
    x = Conv2D(16, (1, min(64, Samples)), padding="same", use_bias=False)(inp)
    x = BatchNormalization()(x)
    x = DepthwiseConv2D((Chans, 1), use_bias=False, depth_multiplier=2,
                        depthwise_constraint=max_norm(1.0))(x)
    x = BatchNormalization()(x); x = Activation("elu")(x)
    x = AveragePooling2D((1, 4))(x); x = Dropout(DROPOUT)(x)
    x = SeparableConv2D(32, (1, 16), padding="same", use_bias=False)(x)
    x = BatchNormalization()(x); x = Activation("elu")(x)
    x = AveragePooling2D((1, 8))(x); x = Dropout(DROPOUT)(x)
    x = Flatten()(x)
    x = Dense(256, activation="elu")(x)
    x = Dropout(DROPOUT)(x)
    x = Dense(embed_dim)(x)
    out = Lambda(lambda t: tf.math.l2_normalize(t, axis=1))(x) # embedding na sferze
    model = Model(inp, out, name="ekoder_eeg")
    model.compile(optimizer=Adam(LEARNING_RATE), loss=CosineSimilarity(axis=1))
    model.summary()
    return model

```

Rys. 25. Enkoder EEG (korpus EEGNet) zakończony znormalizowanym wektorem cech oraz jego uczenie z funkcją straty opartą na podobieństwie kosinusowym.

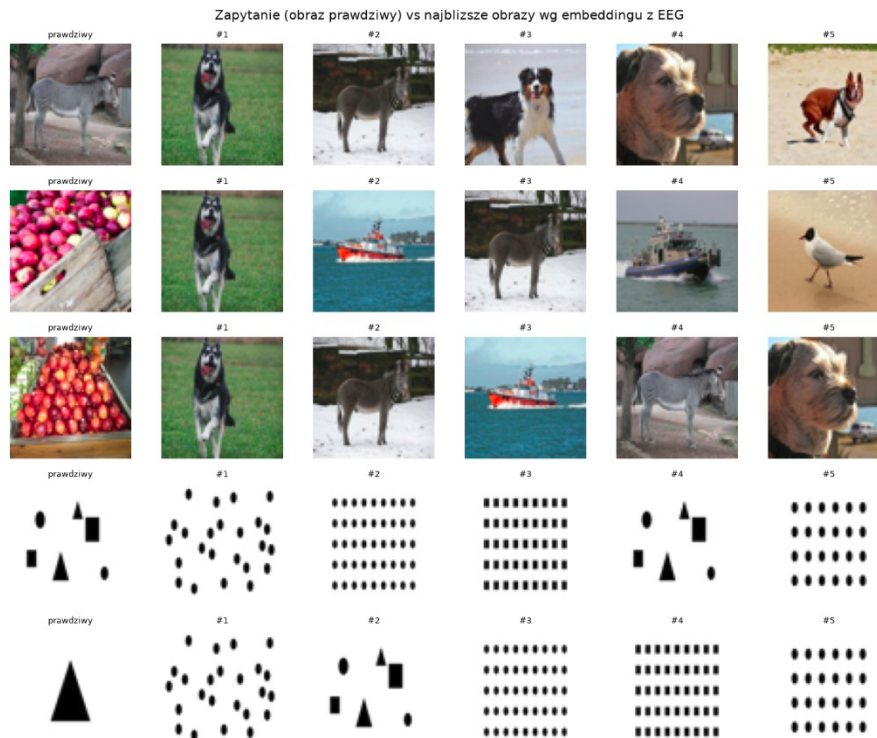
Po wytrenowaniu enkodera rekonstrukcję formułowano jako zadanie identyfikacji. Dla każdej próby testowej wyznaczano wektor reprezentujący sygnał EEG, a następnie porównywano go, za pomocą podobieństwa kosinusowego, z wektorami wszystkich 220 obrazów. Jako wynik identyfikacji wybierano obraz o najwyższym podobieństwie, stosując metodę najbliższego sąsiada (ang. *nearest neighbour*). Zadanie nie sprowadzało się zatem do klasyfikacji obrazu do jednej z jedenastu kategorii, lecz polegało na wskazaniu konkretnego obrazu spośród całego zbioru dostępnych możliwości.

Skuteczność identyfikacji oceniano za pomocą dwóch grup miar. Wskaźniki *top-1* oraz *top-5* określały, czy właściwy obraz znalazł się odpowiednio na pierwszym miejscu lub w pierwszej piątce rankingu spośród 220 obrazów. Identyfikacja *n-way* (dla $k = 2, 5$ oraz 10) określała natomiast, czy właściwy obraz uzyskał wyższe podobieństwo niż $k - 1$ losowo dobranych obrazów pełniących rolę rozpraszaczy. Odpowiadało to zadaniu wyboru właściwego obrazu spośród k możliwości. Procedurę powtarzano wielokrotnie, a następnie uśredniano uzyskane wyniki. Kod implementujący tę miarę przedstawiono na rys. 29. Ocenę przeprowadzono dwukrotnie: dla pełnego zbioru kategorii oraz po usunięciu najłatwiejszej kategorii *abstract*. Pozwoliło to sprawdzić, czy uzyskiwana informacja dotyczy również samych zdjęć, a nie wyłącznie różnic pomiędzy kategoriami.

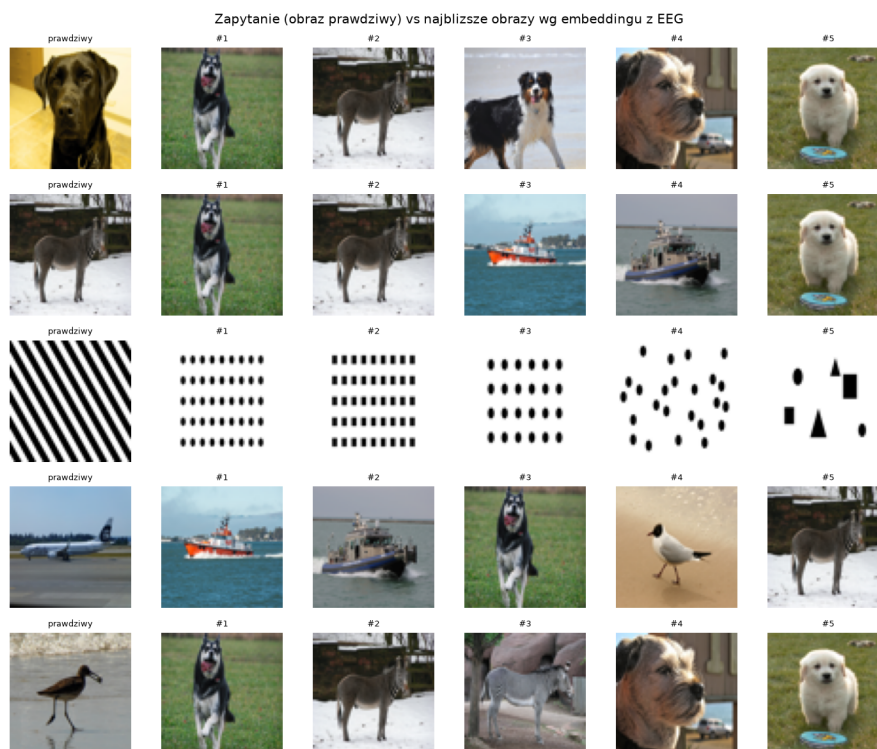
```
# ----- METRYKI -----
def n_way_identyfikacja(sims, true_idx, k, powtorz, seed=0):
    """Srednia trafność: czy poprawny obraz ma wyższe podobieństwo niż k-1 losowych."""
    rng = np.random.default_rng(seed)
    Nt, M = sims.shape
    wygrane, razem = 0, Nt * powtorz
    for i in range(Nt):
        ti = int(true_idx[i])
        cs = sims[i, ti]
        inne = sims[i][np.arange(M) != ti]          # (M-1,)
        idx = rng.integers(0, M - 1, size=(powtorz, k - 1))
        maxd = inne[idx].max(axis=1)                # najlepszy rozpraszacz w każdym losowaniu
        wygrane += int(np.sum(cs > maxd))
    return wygrane / razem
```

Rys. 26. Miara identyfikacji *n-way*: sprawdzenie, czy właściwy obraz uzyskuje wyższe podobieństwo niż $k - 1$ losowych obrazów pełniących rolę rozpraszaczy.

Przykładowe wyniki wyszukiwania, obejmujące obraz-zapytanie oraz pięć najbliższych obrazów w przestrzeni embeddingów, przedstawiono na rys. 30 i 31.



Rys. 27. Identyfikacja: obraz-zapytanie (pierwsza kolumna) oraz pięć najbliższych obrazów według podobieństwa embeddingów uzyskanych z sygnału EEG.



Rys. 28. Identyfikacja: obraz-zapytanie (pierwsza kolumna) oraz pięć najbliższych obrazów według podobieństwa embeddingów uzyskanych z sygnału EEG przy wykluczeniu klasy *abstract*.

6.3. Wyniki i wnioski

Wyniki zestawiono w tabeli 2. Wszystkie miary znalazły się wyraźnie powyżej poziomu losowego – przykładowo identyfikacja 2-way osiągnęła ok. 61%, a 10-way ok. 19% (wobec odpowiednio 50% i 10% dla przypadku). Co istotne, przewaga nad poziomem losowym utrzymała się także po wykluczeniu kategorii *abstract* (2-way ok. 56%, 10-way ok. 15%), choć była wtedy mniejsza. Oznacza to, że sygnał niesie informację nie tylko o łatwym rozróżnieniu grafiki od zdjęć, lecz – w ograniczonym stopniu – także o samych zdjęciach.

Miara	Wszystkie kategorie	Tylko zdjęcia	Poziom losowy
top-1	0,97%	0,85%	ok. 0,5%
top-5	5,45%	4,45%	ok. 2,3%
2-way	61,1%	56,4%	50%
5-way	31,5%	26,7%	20%
10-way	18,8%	14,8%	10%

Tabela 2: Skuteczność identyfikacji obrazów z EEG dla wszystkich kategorii oraz z pominięciem kategorii *abstract*.

Analiza jakościowa ujawniła jednocześnie zjawisko tzw. węzłów (ang. *hub*): kilka obrazów (m.in. przedstawiających psy) pojawiało się jako najbliższy sąsiad wielu różnych zapytań. Świadczy to, że mimo statystycznej przewagi nad poziomem losowym pojedyncze dopasowania pozostawały zaszumione, a *embeddings* zdjęć skłonne były do zbijania się w niewielkim obszarze przestrzeni cech.

Podsumowując, rekonstrukcja pikselowa pojedynczej próby EEG okazała się niemożliwa – model odtwarzał jedynie uśredniony obraz zbioru. Identyfikacja wykazała natomiast, że w sygnale obecna jest słaba, lecz statystycznie istotna informacja o oglądanym obrazie, sięgająca poza samo rozróżnienie grafiki geometrycznej. Wyniki obu podejść są spójne z rezultatami klasyfikacji: informacja o obrazie w sygnale EEG istnieje, jest jednak zbyt słaba do dokładnego, jednostkowego odtworzenia.

7. Podsumowanie projektu - uwagi i wnioski

7.1. Wnioski

Najważniejszym wnioskiem jest to, że sygnał EEG niesie pewną, powtarzalną informację o oglądanym obrazie. Świadczy o tym spójność trzech niezależnych eksperymentów: celność klasyfikacji utrzymywała się powyżej poziomu losowego, podobnie jak identyfikacja obrazów w zadaniu rekonstrukcji. Informacja ta jest jednak słaba – nie pozwala na precyzyjne wskazanie ani odtworzenie konkretnego obrazu.

Kluczowym problemem ograniczającym skuteczność okazały się różnice w sygnałach zebranych dla poszczególnych osób. Przy podziale danych rozłącznym względem badanych (ocenie na osobach nieobecnych podczas treningu) skuteczność spadała niemal do poziomu losowego, natomiast przy podziale mieszanym wyraźnie go przewyższała. Oznacza to, że modele uczyły się wzorców w dużej mierze specyficznych dla poszczególnych osób. Oznacza to, że dwie różne osoby reagują na ten sam obraz odmiennie, co utrudnia zbudowanie modelu uniwersalnego, niezależnego od osoby badanej.

Porównanie architektur i reprezentacji wskazuje ponadto, że ograniczeniem nie jest sam model, lecz ilość informacji zawarta w sygnale. Sieci działające na surowym sygnale nie potrafiły dopasować się do danych treningowych powyżej ok. 22%, a zwiększanie ich pojemności nie poprawiało wyniku, co wskazuje na niedouczenie wynikające z danych. Dodatkowo reprezentacja czasowo-częstotliwościowa oraz sygnał surowy dały porównywalne rezultaty, mimo znacznych różnic w architekturze.

Najlepiej rozpoznawaną kategorią był *abstract* – prosta grafika geometryczna, wizualnie skrajnie odmienna od zdjęć – a w dalszej kolejności *person*. Pozostałe kategorie rozróżniano znacznie słabiej. Sugeruje to, że sygnał odzwierciedla przede wszystkim ogólne, niskopoziomowe cechy bodźca, a nie jego szczegółową treść.

7.2. Uwagi i możliwe kierunki dalszych prac

Pojedyncza próba EEG cechuje się niskim stosunkiem sygnału do szumu. Ponieważ każdy obraz prezentowano sześciokrotnie, uśrednienie powtórzeń tego samego bodźca mogłoby istotnie poprawić jakość sygnału – kosztem liczby przykładów uczących – i jest naturalnym kierunkiem dalszych prób. W kontekście zmienności międzyosobowej obiecujące byłyby natomiast modele uczone lub dostrajane indywidualnie dla każdej osoby, a także zwiększenie liczby badanych.

Kilka elementów potoku również pozostawia pole do usprawnień. Na etapie czyszczenia sygnału składowe artefaktowe (ICA) wykrywano automatycznie – ich ręczna weryfikacja mogłaby poprawić jakość danych. Zbiory uczący i testowy normalizowano niezależnie; w docelowym postępowaniu warto normalizować zbiór testowy statystykami wyznaczonymi na zbiorze uczącym, aby uniknąć podglądania danych testowych. W zadaniu rekonstrukcji rozważyć można z kolei inne funkcje straty (np. perceptualną) oraz metody ograniczające zjawisko węzłów w przestrzeni cech.

Mimo umiarkowanych wyników liczbowych projekt pozwolił przejść pełną ścieżkę badawczą – od surowego sygnału EEG po ocenę modeli – oraz wyciągnąć czytelne wnioski na temat możliwości i ograniczeń dekodowania obrazów z aktywności mózgu.